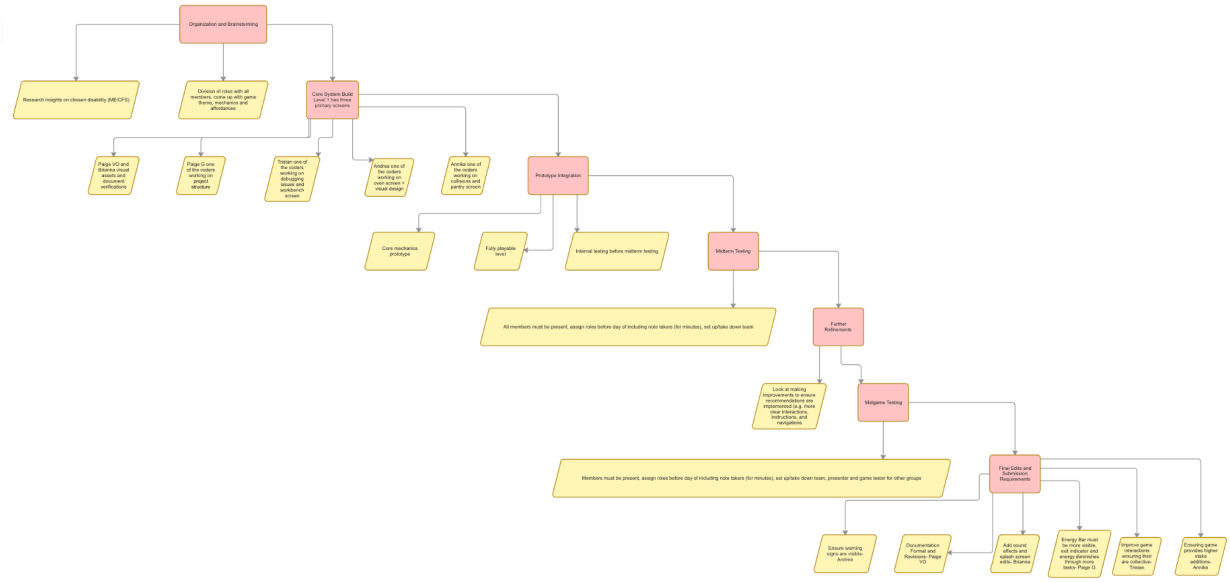
Process & Decision Documentation

Project/Assignment Decisions

This section of the project required us to create our Mid-Term Game, meaning one level is completed. Our team focused on myalgic encephalomyelitis/chronic fatigue syndrome and after learning more about this disability, facts about its unpredictability, and long lasting effects towards energy loss stood out. This knowledge allowed us to create the game Bread and Balance.

Process Overview Visualization



Link to view further:
https://miro.com/app/board/uXjVG7ILgo4=/?share_link_id=433881716990

Role-Based Process Evidence

Game Level 1

Name, role(s), & primary responsibility for this work:

Name	Role	Description
Paige Van Overberghe	Documentation Lead, readme, visual asset collection	Track the group's decisions, iterations, and changes, help with organizing the repo, and assist with coding as needed.
Andrea Sobiesiak	Visual Design, Organization, oven screen	Replace placeholders with polished visuals and design game colours, shapes, and themes. Improve game layout and clarity of game mechanics.
Paige Gugeler	Project Structure + Core Input Coding Lead, main, home, navbar, recipe, end, and sleep screens	Set up all code files correctly and structure modules, write keyboard input systems to ensure player interaction is seamless.
Annika Zamora	Collision & Interaction Lead, pantry screen	Program collision detection, program object interactions within the game, and test the physics feel to make smooth gameplay.
Tristan Whalen	Coding Debugger, Systems Support Lead, workbench screen	Review code to make sure the game runs smoothly while keeping consistent formatting across files. Help out with any specific coding needs.
Brianna Zampini	Visual Design + Audio Lead	Create visual aspects, background music + sound effects, and add audio triggers in code. Ensure any pre-existing assets are properly cited in the reference section.

Goal of Work Session

Complete one level of our video game with basic functions and game mechanics, and include visuals and background music.

Tools, Resources, or Inputs Used

- ChatGPT, Copilot, Claude Haiku 4.5 (built in Visual Studio Code), Adobe Firefly Image Model 5
- Week 3 example code (from LEARN)
- Feedback and notes collected from playtesting

GenAI Documentation

Date: March 5th-9th

Tool Used: Adobe Firefly Image Model 5

Original Prompts:

Prompt #1: We are creating a coding game and require visual assets that are realistic yet cozy to simulate a bakery. We have included reference photos we have drawn regarding the image sizing and minimalistic style. Please generate components of salt, water, flour, sugar, and starter.

Refined prompt #1: The images of those products are what we are looking for. Please keep the same design aspects but separate each object apart from each other rather than clumped together.

Prompt #2: Keeping the same object colours, please provide an overhead shot of the objects lined up.

Refined prompt #2: The objects require text on the top and far side, to ensure the text is clear and legible. Objects from left to right that require labelling are starter, salt on the top and flour on the bottom of the bag.

Prompt #3: Please create a pastel pink bowl with an empty standing up view.

Refined prompt #3: Keep the pink bowl image the exact way please add in realistic uncooked bread filling to the top of the bowl.

Prompt #4: Please have the same bowl right side up and include the uncooked bread within.

Refined prompt #4: The bowl requires a realistic view of the bread, instead of a flat circle please add volume to the bread's uncooked texture.

Prompt #5: Create a ceramic cooking pot with cooked bread to ensure there are two handles.

Refined prompt #5: Create a similar ceramic cooking pot, instead of bright blue please make it a dark blue to complement the bakery colours already chosen for the bowl and ingredients.

Prompt #6: Create the same ceramic cooking pot but insert uncooked bread with a simplistic design on the bread.

Refined prompt #6: Please keep the same design ceramic pot and bread, however add a lid on the edge of the pot.

Prompt #7: Create an image of the ceramic cooking pot with the lit over top.

Prompt #8: Create a ceramic cooking pot with cooked bread to ensure there are two handles.

Refined prompt #8: Create a similar ceramic cooking pot, instead of bright blue please make it a dark blue to complement the bakery colours already chosen for the bowl and ingredients.

Prompt #9: Please create a countertop with draws and the bowl in the centre. We are looking for pale blue on these drawers.

Prompt #10: Please create the counter top from an overhead review and remove the blue bowl and replace it with the pink bowl already created.

Prompt #11: Create an open cabinet that is pale blue, empty and provides through levels.

Refined Prompt #11: Please keep the same cabinet design, remove ingredients to keep it empty and change to blue instead of pink.

Prompt #12: Create an open oven that is corresponding to the already chosen colour including details such as lighting, realistic buttons and stove tops.

Purpose of Use: We provided Firefly with reference images we had drawn previously and to ensure time efficiency we used the tool to generate more visual assets for our bakery layout. When things became complex we would refine the prompts to create visuals we were satisfied with.

Summary of AI conversation: We asked the tool to generate images for a bakery scene, asked for refinements and additions on each image chosen.

Human Decision Point(s):

We looked at refining the prompts to provide better accuracy and more variety for our groupmates who are coding these visuals for our game. This was important as they also asked for a few changes within the visuals, they asked to have all the ingredients

scattered throughout the pantry. Therefore, we chose to take the images and place the ingredients in a less chaotic manner. Firefly originally misplaced various objects where our coders did not want them.

From a design perspective, we kept options blank and had our coders ask us to place various visuals onto the countertops as well in a specific manner that resonated with their coding format. It was an easy solution for the drag and drop feature. We also decided to remove the blue bowl on the counter (Prompt #10) as we believed the pink bowl gave a better complementary colour for the bakery.

Integrity & Verification Note: We cross-verified with our reference images, consulted with our team through our process of collecting images to ensure it matched the aesthetics everyone was looking for.

Scope of GenAI Use: Generated realistic bakery visuals from reference images allowing for an easy transition into our game.

Limitations or Misfires: Overall, the tool was very helpful and did not have many misfires as we provided reference images and when these settings are added Firefly recognizes what images resemble best. Firefly did misplace certain ingredients and missed certain elements that go with objects to complete each step of the baking process.

Prompt Authors: Paige Van Overberghe and Brianna Zampini

Date Used: March 7th-9th

Tool Disclosure: Claude Haiku 4.5

Purpose of Use: I used GenAI to help me fix bugs in my code that I could not work out myself, and complete complex code.

Summary of Interaction: The tool completed small additions to my code to ensure functionality. I started by coding everything myself and then asked AI to help me fix small sections so the code would work as intended.

Human Decision Point(s): Since I created all the code to start with, I decided how I wanted everything to work and look. GenAI helped me complete my vision by working out the things I didn’t know how to do. I then modified the GenAI code in a few places as I noticed it would do something very inefficiently and I realized I could do the same

thing in a simpler way. The AI output acted as a starting point that provided me with the idea and functions I needed.

Integrity & Verification Note: I read over all the GenAI code to ensure it was doing what I wanted in an efficient way. I also checked the visual output and clicked on everything to find any issues it created.

Scope of GenAI Use: GenAI contributed to the navigation bar and auto-playing the introduction video. All the screens and base code were completed by me, but I had issues layering these features so GenAI helped me implement and debug them.

Limitations or Misfires: The output would occasionally be very obviously inefficient so I would rewrite the code in a simpler way. It also claimed to fix some things that it did not fix so I had to redo some code.

Prompt Authors: Paige Gugeler

Date Used: March 9-12

Tool Disclosure: Visual Studio Code Copilot Add-On, ChatGPT 5.2 (Pro)

Purpose of Use: I used GenAI to write/modify the game concepts and mechanics into functional code. I mainly used it to build core functions and mechanics, then adjusted it afterwards without the use of GenAI

Summary of Interaction: The prompts first started by including uploaded files of code, or sections of code that were copied and pasted in. I then wrote 1-3 sentences that described what I wanted the code to execute (ex, new function to make the flour bag clickable, a counter for the number of ingredients). The prompts were specific to small areas of code so that it could focus on one thing at a time and allowed for more detail to be put into each section.

Human Decision Point(s): I started off using the Copilot that was a part of Visual Studio Code. It worked for simpler changes and prompts such as locating the part of the code that ran a certain function, however when I prompted for more complicated mechanics and functions, it started having issues and wouldn’t provide an output that worked/was what I was looking for. I decided to switch over to ChatGPT Pro (online in a separate tab) and the outputs worked much better.

Integrity & Verification Note: After every output/suggested change, I tested the code by running it and seeing if it functioned the way I was intending for it to. For most

prompts, it built a simple foundation, where I could then adjust some of the details afterwards (ex, adjusting x and y coordinates for a button).

Scope of GenAI Use: GenAI did not create the concepts for any of the functions, mechanics, and ideas behind the game. GenAI was only responsible for translating those ideas into functioning code. All prompts were written without AI.

Limitations or Misfires: When first using Copilot, it often modified code and said it followed the prompt, however when I ran it, it was either not appearing as it said it did, or the code did not run. I also found that visual studio code predicts and fills in areas of the code, however sometimes it wouldn't predict it correctly which would slow down the work process, or become confusing.

Prompt Authors: Annika Zamora

Date Used: March 5-10th

Tool Disclosure: Claude Sonnet 4.6

Purpose of Use: I used AI to help me code our workbench. I used it to add images so it looked like a bread bakery workbench. Also I used it to code click to add ingredients, click to bake bread and made it so when you're done and you press bake bread it brings you directly to the oven.

Summary of Interaction: Claude Sonnet 4.6 did all the coding for me. I suggested ideas to make it look better visually but Clause really brought my ideas to life.

Human Decision Point(s): It would create the workbench the way it wanted to but I had to keep it in line with our overall design requirements and our unique style. I also had to make sure there were no errors by skimming through code to look for any mistakes.

Integrity & Verification Note: I checked the accuracy of the AI by letting peers evaluate the code and going over it myself. Also Github is good at telling you where errors are if you check the console.

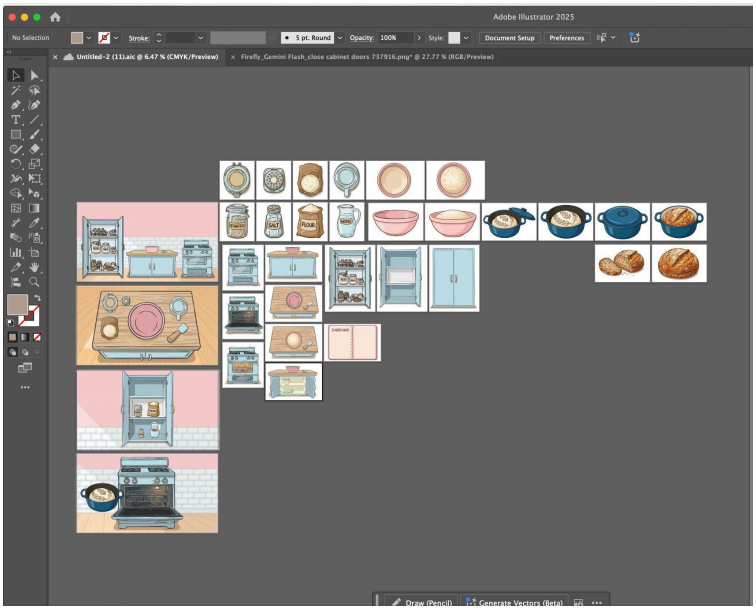
Scope of GenAI Use: The images that were incorporated were my decision and I added those in personally. Also my group suggested certain styles that I went in and changed to please my team such as size of workbench bowl and bake bread button. I did this by editing the code to my liking.

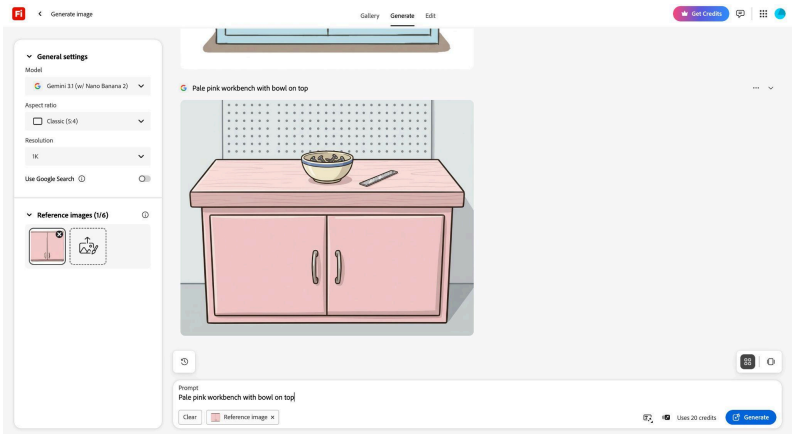
Limitations or Misfires:AI has a hard time being creative without a sense of direction. That's where I came in and shined brighter than the AI.

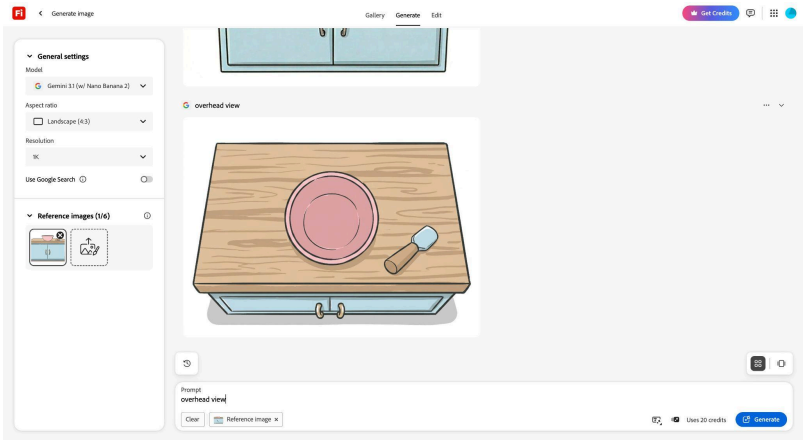
Prompt Authors: Tristan Whalen

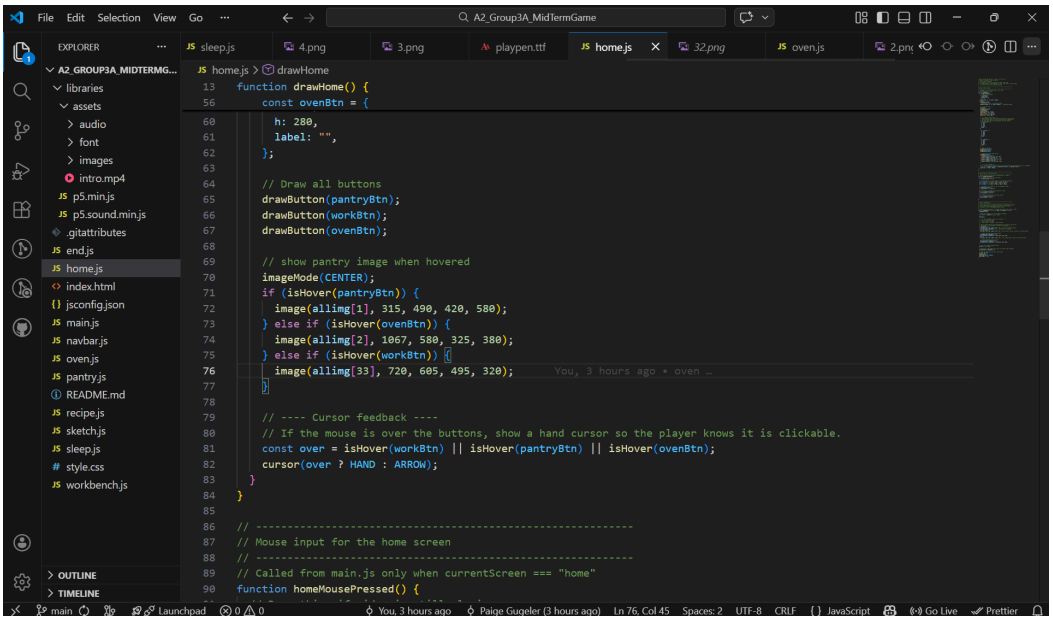
Summary of Process

1. The group roles were assigned based on group member strengths
2. We brainstormed game ideas collectively
3. Paige V.O. and Brianna created the game visuals using Adobe Illustrator, Indesign, and Firefly

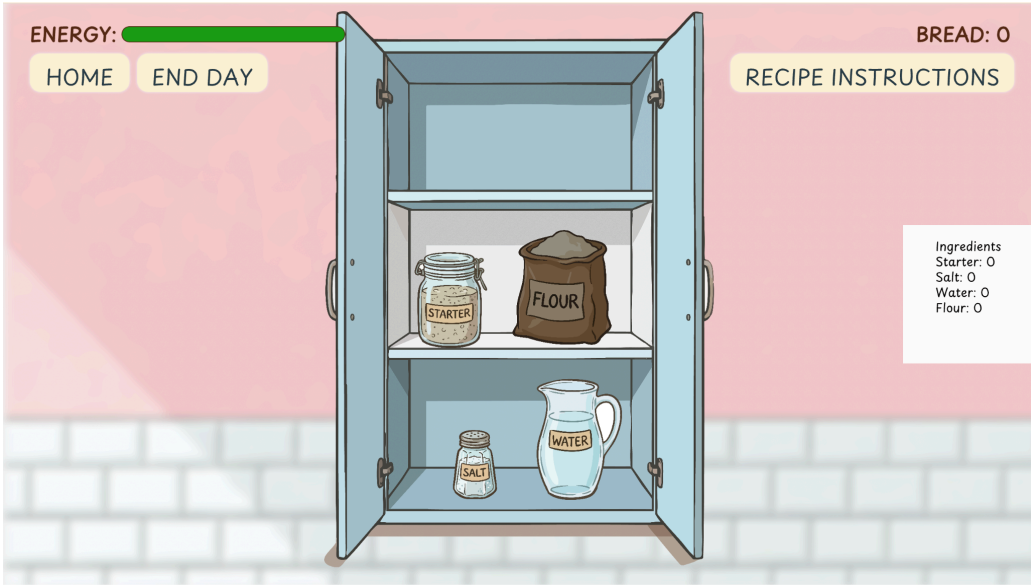


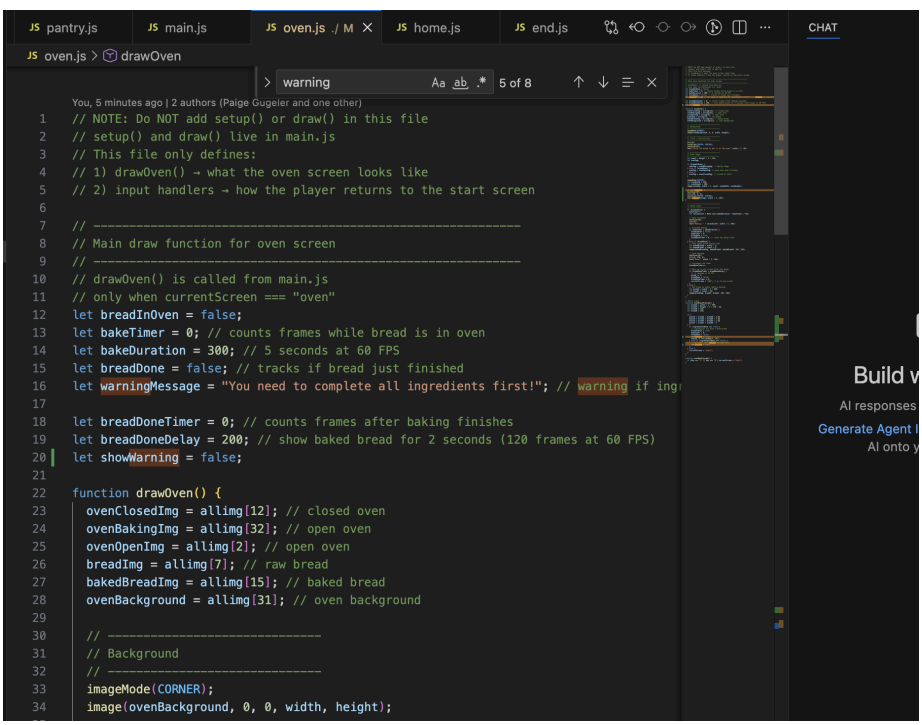


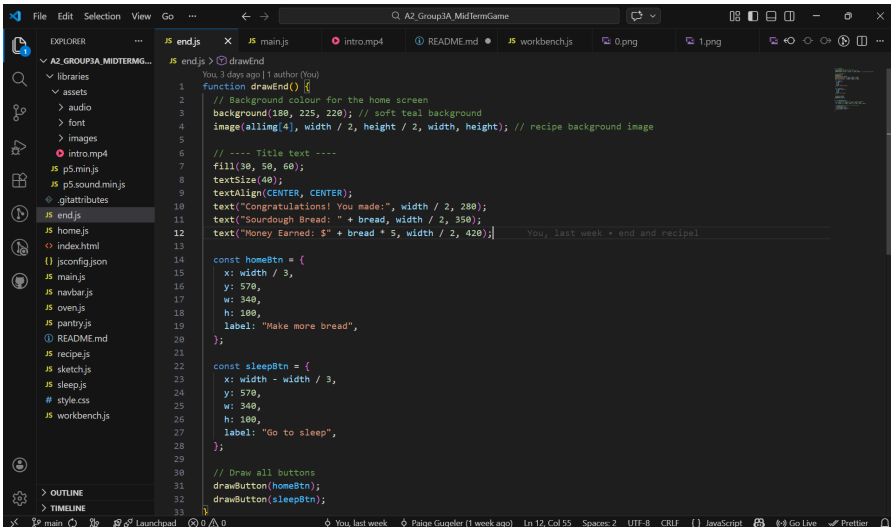


4. Paige G. set up and linked the code files based on the Week 3 example code and preloaded all the visual assets
    a. Ensuring everything appears in index and the draw, mouse, and key pressed functions are called accordingly
5. Paige G. coded and implemented the visuals for the main, home, and recipe screens, and the navigation bar
    a. Creating the buttons to link between screens
    b. Ensured hovering caused the appropriate visuals to show
    c. Set up the energy and bread counters
    d. 

6. Annika coded and implemented the visuals for the pantry screen
    a. Made each image darker when cursor hovered over

b. Updated the counter for each ingredient to increase by 1 after user clicks on it
c. Added a counter on the side to display how much of each ingredient was collected
d. 

7. Andrea coded and implemented the visuals for the oven screen
   a. Coded baking the bread, ensuring the user can not bake unless they have completed the ingredients, and get a warning if they attempt to. It also changes images from closed to baking to open oven.
   b. 

8. Tristan coded and implemented the visuals for the workbench screen
    a. Coded the workbench to add all ingredients together before the baking process. Each click of a button costs energy to mix ingredients and you must click each ingredient to add it to the bowl. The image of the bowl changes when all ingredients are in the bowl. The trash button scraps bread. The bake bread button works when all ingredients are in the bowl and sends you to the oven.
9. Paige G. coded the end and sleep screens
    a. 

```
function drawEnd() {
  // Background colour for the home screen
  background(180, 225, 220); // soft teal background
  image(allimg[4], width / 2, height / 2, width, height); // recipe background image

  // ---- Title text ----
  fill(30, 50, 60);
  textSize(40);
  textAlign(CENTER, CENTER);
  text("Congratulations! You made:", width / 2, 280);
  text("Sourdough Bread: " + bread, width / 2, 350);
  text("Money Earned: $" + bread * 5, width / 2, 420);

  const homeBtn = {
    x: width / 3,
    y: 570,
    w: 340,
    h: 100,
    label: "Make more bread",
  };

  const sleepBtn = {
    x: width - width / 3,
    y: 570,
    w: 340,
    h: 100,
    label: "Go to sleep",
  };

  // Draw all buttons
  drawButton(homeBtn);
  drawButton(sleepBtn);
}
```

10. Annika and Paige V.O. took notes during playtesting and we then reassigned tasks as a group

Notes collected during testing

- Include instructions tab throughout the whole game so players can reference back to them
- Ingredients were able to drag over (workbench)
- When the right amount of ingredients are put in pages work well*
- **Should we make sure the oven is not allowed to be used until the workbench is complete? yes**
- Put instructions within the game even after the introduction states the objectives
- Users were confused on how much of each ingredient to get → had to rewatch intro
- Users seemed unsure at some points, especially at the beginning stages of the game
- No flour in pantry
- Navigation between pages seemed unclear
- Skipped from the pantry to the oven (didn't reach the workbench)
- Possibly show the recipe card on the work bench in the recipe tab?
- What happens when a player makes an error? (adding too much of an ingredient in the bowl, what do they do after?)
- Not sure what energy levels have to do with game/how it behaves in the game
- May need bigger decrease rate in energy

Changes needed for next stages of game:
Paige G:

- [x] ~~Zero energy = done for the day~~
- [x] ~~Change energy level from number to a bar (green full, yellow middle, red low)~~
- [x] ~~Energy decreases in larger amounts~~
- [x] ~~Specify/brief description what each button is e.g. workbench~~
- [x] ~~Include rules/instructions throughout the game not just in the intro~~

    a. **Annika:**

11. Annika, Tristan, Andrea, and Paige G. modified the code based on the playtesting feedback

Decision Points & Trade-offs

1. One of the key decisions our group made was to decrease the energy by a randomized amount on every click and to not show a number of the decrease. Originally we had a counter instead of an energy bar, but after considering the disability and the fact that it is unrealistic for a human to know an exact percentage of how much energy they have or how much energy a task will take, we decided the bar would make it more ambiguous and aligned with the disability. After receiving feedback during playtesting, we chose to use colours to indicate roughly how much energy they have. This information is realistic for a human to know about themselves, and then players can use it to understand and inform their game decisions (baking more bread vs going to sleep).
2. Another key decision we made as a group was allowing room for user error. This was displayed through game mechanics such as allowing players to collect more ingredients than needed, and allowing them to put as many ingredients as they collected into the bowl. This created more challenge in the game by not restricting the players, which meant they had to figure out and follow the recipe closely in order to succeed. Trial and error is also an effective way for players to understand a game they haven't played before, and also make decisions within the game.

Verification & Judgement

- Running the code after each modification to test it
- Communicated ideas and problems with other group members
- Completed the code incrementally (based on individual game screens) so that we could quickly debug and add/remove features
- Referred to course concepts from LEARN

Limitations, Dead Ends, or Open Questions

- We only created one level, so there is currently no end screen/ending for the game
- Currently only one type/recipe of bread to keep it simple for the first level

Appendix

GenAI Transcript

Prompt	**GenAI Output**

Andrea - Show“Add code to make the bread move to the oven when it is clicked:
 	
function drawOven() {
 	
background(200, 255, 200);
 	
fill(0);
textAlign(CENTER, CENTER);
 	
textSize(40);
text("oven!", width / 2, 50);
 	
textSize(20);
text("Click or press R to return to Start.", width / 2, 90);
 	
rectMode(CENTER);
fill(150);
let cornerRadius = 30;
rect(width / 2, height / 2, 400, 300, cornerRadius);
fill(200, 150, 100); // tan color
rectMode(CENTER);
rect(width / 2 + 300, height / 2, 100, 60, 20); // x, y, width, height, corner radius
}”
	“Absolutely! We can do this **super simple** by using a single **boolean variable** that just tracks whether the bread has been “put in the oven.” No objects, no complicated movement — just draw the bread in a new position if the player clicked it.
	
	Here’s the simplest way to do it in your `oven.js` file:
	
	// Tracks whether the bread is in the oven
	let breadInOven = false;
	
	// ----------
	// Main draw function for oven screen
	// ----------
	function drawOven() {
	// Green-tinted background
	background(200, 255, 200);
	
	fill(0);
	textAlign(CENTER, CENTER);
	
	// Main success message
	textSize(40);
	text("oven!", width / 2, 50);
	
	// Instruction text
	textSize(20);
	text("Click the bread to put it in the oven, or press R to return.", width / 2, 90);
	
	// ----------
	// Draw the oven
	// ----------
	rectMode(CENTER);
	fill(150);
	let cornerRadius = 30;
	rect(width / 2, height / 2, 400, 300, cornerRadius);
	
	// ----------
	// Draw the bread
	// ----------
	fill(200, 150, 100); // tan bread color

```
 rectMode(CENTER);

 if (breadInOven) {
   // If bread clicked, draw it inside the oven
   rect(width / 2, height / 2, 100, 60, 20);
 } else {
   // Otherwise, draw it to the right of the oven
   rect(width / 2 + 300, height / 2, 100, 60, 20);
 }
}

// ------------------------------------------------------------
// Mouse input for oven screen
// ------------------------------------------------------------
function ovenMousePressed() {
 // Check if the click is roughly on the bread on the right
 let breadX = width / 2 + 300;
 let breadY = height / 2;
 let breadW = 100;
 let breadH = 60;

 if (
   mouseX > breadX - breadW / 2 &&
   mouseX < breadX + breadW / 2 &&
   mouseY > breadY - breadH / 2 &&
   mouseY < breadY + breadH / 2
 ) {
   breadInOven = true; // bread is now in the oven
 } else {
   currentScreen = "start"; // clicking anywhere else returns to start
 }
}

// ------------------------------------------------------------
// Keyboard input for oven screen
// ------------------------------------------------------------
function ovenKeyPressed() {
 if (key === 'r' || key === 'R') {
   currentScreen = "start";
 }
}
```

✅ **How it works (super simple version):**

	1. Bread starts **to the right of the oven**. 2. If the player clicks it, `breadInOven = true`. 3. If `breadInOven` is true, the bread is drawn **inside the oven**. 4. Clicking anywhere else or pressing `R` returns to the start screen. This avoids any complicated animations or objects — it's just a **simple toggle**.

Andrea - "Now is there any way to extremely simply code a timer so the bread stays in the oven for five seconds once its clicked and then its done and goes back to its original spot?"	Nope — this can be done **super simply** using just a **counter variable**. We don't need fancy timers or objects. We just count frames in `drawOven()` and reset after 5 seconds.
	
	Assuming your sketch runs at the default **60 frames per second**, 5 seconds = 60 × 5 = 300 frames.
	
	Here's an **extremely simple way**:
	
	let breadInOven = false;
	let bakeTimer = 0; // counts frames while bread is in oven
	
	function drawOven() {
	// Green-tinted background
	background(200, 255, 200);
	
	fill(0);
	textAlign(CENTER, CENTER);
	
	textSize(40);
	text("oven!", width / 2, 50);
	
	textSize(20);
	text("Click the bread to put it in the oven, or press R to return.", width / 2, 90);
	
	// Draw oven
	rectMode(CENTER);
	fill(150);
	rect(width / 2, height / 2, 400, 300, 30);
	
	// Draw bread

```
 rectMode(CENTER);
 if (breadInOven) {
   fill(200, 120, 50); // darker color while baking
   rect(width / 2, height / 2, 100, 60, 20);

   // Increase timer
   bakeTimer++;

   // After 5 seconds (300 frames), reset bread
   if (bakeTimer > 300) {
     breadInOven = false;
     bakeTimer = 0;
   }
 } else {
   fill(200, 150, 100); // raw bread color
   rect(width / 2 + 300, height / 2, 100, 60, 20);
 }
}

// Mouse input
function ovenMousePressed() {
 // Click on bread to start baking
 let breadX = width / 2 + 300;
 let breadY = height / 2;
 let breadW = 100;
 let breadH = 60;

 if (
   mouseX > breadX - breadW / 2 &&
   mouseX < breadX + breadW / 2 &&
   mouseY > breadY - breadH / 2 &&
   mouseY < breadY + breadH / 2
 ) {
   breadInOven = true;
   bakeTimer = 0;
 } else {
   currentScreen = "start";
 }
}

// Keyboard input
function ovenKeyPressed() {
 if (key === 'r' || key === 'R') currentScreen = "start";
```

	}

Andrea - “Show how I can add images to replace the shapes in this code: let breadInOven = false;
let bakeTimer = 0; // counts frames while bread is in oven
let bakeDuration = 300; // 5 seconds at 60 FPS
let breadDone = false; // tracks if bread just finished
 	
function drawOven() {
  // Green-tinted background
  background(200, 255, 200);
 	
  fill(0);
  textAlign(CENTER, CENTER);
 	
  textSize(40);
  text("oven!", width / 2, 50);
 	
  textSize(20);
  text("Click the bread to put it in the oven, or press R to return.", width / 2, 90);
 	
  // Draw oven
  rectMode(CENTER);
  fill(150);
  rect(width / 2, height / 2, 400, 300, 30);
 	
  // Draw bread
  rectMode(CENTER);
  if (breadInOven) {
  fill(200, 120, 50); // darker color while baking
	“To replace the **rectangles with images** in p5.js, you only need to do **three simple things**:
	1. **Load the images in `preload()`**
	2. **Create variables to store the images**
	3. **Replace `rect()` with `image()`**
	
	Since your file says `setup()` and `draw()` live in **main.js**, the images should also be loaded there.
	--- 	
	1. Add image variables (top of main.js)
	At the top of your main file:
	`let ovenImg;`
	`let breadRawImg;`
	`let breadBakingImg;`
	--- 	
	2. Load the images in `preload()`
	`function preload() {`
	`  ovenImg = loadImage("assets/oven.png");`
	`  breadRawImg = loadImage("assets/raw_bread.png");`
	`  breadBakingImg = loadImage("assets/baking_bread.png");`
	`}`
	
	Your images should be in:
	`project folder`
	`  ├─ assets`
	`  |  ├─ oven.png`
	`  |  ├─ raw_bread.png`
	`  |  └─ baking_bread.png`
	--- 	
	3. Replace the oven rectangle
	Replace this:
	`rect(width / 2, height / 2, 400, 300, 30);`
	
	with:
	`imageMode(CENTER);`
	`image(ovenImg, width / 2, height / 2, 400, 300);`

```
    rect(width / 2, height / 2, 100, 60, 20);

    // Increase timer
    bakeTimer++;

    // Draw countdown in seconds
    let secondsLeft = Math.ceil((bakeDuration - bakeTimer) / 60);
    textSize(30);
    fill(0);
    text("Baking: " + secondsLeft, width / 2, height / 2 - 100);

    // After baking is done
    if (bakeTimer > bakeDuration) {
      breadInOven = false;
      bakeTimer = 0;
      breadDone = true; // mark bread as done
    }
  } else {
    fill(200, 150, 100); // raw bread color
    rect(width / 2 + 300, height / 2, 100, 60, 20);
  }

  // ------------------------------
  // Show "Done!" at the top if bread finished
  // ------------------------------
  if (breadDone) {
    textSize(50);
    fill(0, 150, 0); // green
    text("Done!", width / 2, 30);
  }
}
```

---

4. Replace the bread rectangles
When bread is baking
Replace this:
`rect(width / 2, height / 2, 100, 60, 20);`

with:
`image(breadBakingImg, width / 2, height / 2, 120, 80);`

When bread is outside the oven
Replace this:
`rect(width / 2 + 300, height / 2, 100, 60, 20);`

with:
`image(breadRawImg, width / 2 + 300, height / 2, 120, 80);`

---

5. That's it
Your shapes are now images.

Small optional improvement
If your bread image has weird alignment, add this **once before drawing images**:
`imageMode(CENTER);`

“I just need to add a variable that checks if the ingredients are collected and if not it does not let the bread be baked”

“**1. Add an ingredients variable**
At the top with your other variables:
`let ingredientsCollected = false;`

When the player collects the ingredients somewhere else in your game, set it to:
`ingredientsCollected = true;`

---

2. Prevent baking if ingredients are missing
Modify your **mouse click check** in `ovenMousePressed()`.
Replace this part:
```
breadInOven = true;
bakeTimer = 0;
```

// Mouse input
function ovenMousePressed() {
  let breadX = width / 2 + 300;
  let breadY = height / 2;
  let breadW = 100;
  let breadH = 60;
 	
  if (
  mouseX > breadX - breadW / 2 &&
  mouseX < breadX + breadW / 2 &&
  mouseY > breadY - breadH / 2 &&
  mouseY < breadY + breadH / 2
) {
  breadInOven = true;
  bakeTimer = 0;
  breadDone = false; // reset "done" message when new bread is baking
  } else {
  currentScreen = "start";
  }
}
 	
// Keyboard input
function ovenKeyPressed() {
  if (key === 'r' || key === 'R') currentScreen = "start";
}”
	`breadDone = false;`
	
	with this:
	`if (ingredientsCollected) {`
	`  breadInOven = true;`
	`  bakeTimer = 0;`
	`  breadDone = false;`
	`}`
	
	--- 	
	3. (Optional but helpful) Show a message
	Inside `drawOven()` you can show a warning if ingredients aren't collected:
	`if (!ingredientsCollected) {`
	`  fill(200, 0, 0);`
	`  textSize(20);`
	`  text("You need to collect ingredients first!", width / 2, height / 2 - 140);`
	`}`
	
	--- 	
	Result
	Now the logic is:
	• `ingredientsCollected = false` → bread **cannot be baked**
	• `ingredientsCollected = true` → bread **can be baked**
	• Everything else works the same.
	
	--- 	
	✅ **Ultra simple logic**
	`if ingredients collected`
	`  allow baking`
	`else`
	`  do nothing`.”

Paige G - make the navbar buttons visible but the home screen buttons transparent	Edited home.js+0-0 The change has been made. Now the navbar buttons (HOME, RECIPE, END DAY) will be fully visible with their background colors, while the home screen buttons (pantry, workbench, oven) will remain transparent since they have no labels. This way, the navbar provides clear visual feedback for navigation, while the home screen buttons are invisible but still clickable based on their hover images.

Paige G - make the buttons grey and unclickable on this screen	The buttons on the sleep screen are now grey and unclickable. Here's what I changed: Modified drawNavbar() in navbar.js: Added a disabled flag that's true when currentScreen === "sleep", and passed it to each drawButton() call. Updated drawButton() in home.js: Added an optional disabled parameter (defaults to false). When disabled, buttons are drawn in grey (fill(200, 200, 200)) with no shadow effects. Modified navbarMousePressed() in navbar.js: Added a check to return early if currentScreen === "sleep", preventing any button clicks from being processed on the sleep screen. The navbar buttons (HOME, RECIPE INSTRUCTIONS, END DAY) will appear grey and won't respond to clicks when the player is on the sleep screen. On other screens, they work normally.
Paige G - play the video and hide the nav bar, then when the video ends show the home game screen	Perfect! I've implemented the functionality you requested. Here's what I changed: Added a videoFinished flag in main.js to track when the intro video has completed. Set up a video end handler in the setup() function that sets videoFinished = true when the video finishes playing. Modified the draw loop to only show the navigation bar after the video has finished. Updated the home screen logic to play the video first, then display the game interface once the video ends. Disabled mouse interactions during the video playback to prevent accidental clicks. Now when the game starts: The intro video plays immediately The navigation bar is hidden during playback Once the video ends, the home game screen appears with the navigation bar visible Players can then interact with the game normally The changes ensure a smooth transition from the introductory video to the main game interface, hiding UI elements during the cinematic experience as requested.
Annika- on the pantry file, draw three shelves (thin rectangles) going across	Three shelves (thin rectangles) are now drawn across the pantry screen. They appear as horizontal bars at 30%, 50%, and 70% of the canvas height. If you want to adjust their position or style, let me know!
Annika- draw a rect to represent the flour bag. should be placed on the shelf.	A rectangle representing the flour bag is now drawn on the first shelf in the pantry screen, labeled "FLOUR." If you want to adjust its position, size, or style, just let me know!

Annika- make it so that the ingredients are clickable (when hovering, cursor changes, shade of rectangle gets a bit darker)	The ingredient rectangles (flour, water, salt) are now clickable: when you hover over them, the cursor changes to a hand and the rectangle gets a bit darker. Clicking each ingredient is now detected in the pantryMousePressed function, where you can add your own logic for each click. Let me know if you want to trigger any specific actions!
Annika- remove the rectangles being drawn for every ingredient	The rectangles for every ingredient have been removed from the drawing code. Only the ingredient labels (text) will now appear. If you want to add a different visual or interactive element, let me know!

Annika- on javascript visual studio code, is there a way to have dimensions for an irregular shape? I have a pdf of a water jug which isn't a rectangle where i could do width and height dimensions. Is there a way to detect the area of a png	Basic idea:
	`const img = new Image();`
	`img.src = "waterjug.png";`
	
	`img.onload = () => {`
	`  const canvas = document.createElement("canvas");`
	`  const ctx = canvas.getContext("2d");`
	
	`  canvas.width = img.width;`
	`  canvas.height = img.height;`
	
	`  ctx.drawImage(img, 0, 0);`
	
	`  const imageData = ctx.getImageData(0, 0, canvas.width, canvas.height);`
	`  const data = imageData.data;`
	
	`  let visiblePixels = 0;`
	
	`  for (let i = 3; i < data.length; i += 4) {`
	`  const alpha = data[i];`
	`  if (alpha > 0) {`
	`  visiblePixels++;`
	`  }`
	`  }`
	
	`  console.log("Visible area in pixels:", visiblePixels);`
	`};`
	How this works:
	● Every pixel has RGBA values
	● The **alpha** value tells you transparency.
	● If `alpha > 0`, that pixel is part of the jug.
	● Counting those pixels gives the shape's area in **pixel units**.
	
	If you want the **bounding box** of only the visible part, you can also find:
	● leftmost visible pixel
	● rightmost visible pixel
	● topmost visible pixel
	● bottommost visible pixel
	
	That gives you the jug's true occupied width and height, instead of the full image

	rectangle. Important note: ● A **PDF** is different from a PNG. If the jug is inside a PDF, you would usually first export it as a PNG with transparency. ● If the PNG has a solid white background instead of transparency, detection is harder, but still possible by checking color instead of alpha. So yes, JavaScript in VS Code can detect the area of an irregular PNG shape. The cleanest method is: ● use a **PNG with transparent background** ● draw it to a **canvas** ● count non-transparent pixels If you want, I can give you: ● a version that finds the jug's exact occupied width/height, or ● a version that works directly in your p5.js game.

Annika- this is my code. how would that apply in it: // NOTE: Do NOT add setup() or draw() in this file // setup() and draw() live in main.js // This file only defines: // 1) drawPantry() → what the pantry screen looks like // 2) input handlers → how the player returns to the home screen // 3) helper functions specific to this screen // ---------- // Main draw function for pantry screen // ---------- // drawPantry() is called from main.js // only when currentScreen === "pantry" function drawPantry() { // Light neutral background background(240); imageMode(CORNER); image(allimg[28], 0, 0, width, height); // draw sourdough starter png image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140); // ---- INGREDIENTS ---- // flour const flourBtn = { w: 100, h: 100, x: width / 2, y: height / 3, label: "Flour",	Yes — in your code, this would mainly replace the current rectangle hitbox check for the PNG with a pixel-based hit test.
	
	Right now, your pantry items use boxes like this:
	
	const waterBtn = { x: width * 0.5, y: height / 3, w: 120, h: 180 };
	
	and then:
	
	if (isHover(waterBtn)) {
	  waterCounter++;
	}
	
	That means the game is checking a rectangle, even if the water jug shape is irregular.
	
	What to do instead
	
	For a PNG with transparency, you can check:
	
	is the mouse inside the image's drawn bounds?
	
	if yes, is the pixel under the mouse opaque or transparent?
	
	If it is opaque, treat it as a click on the jug.
	
	Add this helper function
	
	Put this somewhere in your file:
	
	function isMouseOverOpaqueImage(img, x, y, w, h) {
	  // first: check if mouse is inside the image bounds
	  if (mouseX < x || mouseX > x + w || mouseY < y || mouseY > y + h) {

}; // water const waterBtn = { w: 120, h: 180, x: width * 0.5, y: height / 3, label: "Water", }; // sourdough starter const starterBtn = { w: 80, h: 140, x: width * 0.25, y: height / 3, label: "Starter", }; // salt const saltBtn = { w: 50, h: 100, x: width * 0.75, y: height / 2, label: "Salt", }; // Draw all buttons drawButton(waterBtn); drawButton(starterBtn); drawButton(saltBtn); drawButton(flourBtn); text(waterCounter, waterBtn.x, waterBtn.y + 20); text(saltCounter, saltBtn.x, saltBtn.y + 20); text(starterCounter, starterBtn.x, starterBtn.y + 20); text(flourCounter, flourBtn.x, flourBtn.y + 20); } // ---------- // Mouse input for instructions screen // ---------- // Called from main.js only when currentScreen === "pantry" function pantryMousePressed() { const backBtn = { x: width / 2, y: 560, w: 220, h: 70 }; const waterBtn = { x: width * 0.5, y: height / 3, w: 120, h: 180 }; const starterBtn = { x: width * 0.25, y: height / 3, w: 80, h: 140 }; const saltBtn = { x: width * 0.75, y: height / 2, w: 50, h: 100 }; const flourBtn = { x: width / 2, y: height / 3, w: 100, h: 100 }; // If the button is clicked, return to the home screen if (isHover(backBtn)) { currentScreen = "home"; return; } //checks if ingredients are clicked if	return false;
	}
	
	// convert mouse position on screen to pixel position inside original image
	let imgX = floor(map(mouseX, x, x + w, 0, img.width));
	let imgY = floor(map(mouseY, y, y + h, 0, img.height));
	
	// keep within bounds
	imgX = constrain(imgX, 0, img.width - 1);
	imgY = constrain(imgY, 0, img.height - 1);
	
	// get pixel color from image
	let c = img.get(imgX, imgY);
	
	// c = [r, g, b, a]
	let alphaValue = c[3];
	
	// treat only visible pixels as clickable
	return alphaValue > 10;
	}
	How that applies to your pantry code
	
	For example, if your water jug PNG is in allimg[whateverIndex], draw it like this:
	
	const waterX = width * 0.5;
	const waterY = height / 3;
	const waterW = 120;
	const waterH = 180;
	
	image(allimg[WATER_INDEX], waterX, waterY, waterW, waterH);
	
	Then in pantryMousePressed() instead of:
	
	if (isHover(waterBtn)) {
	waterCounter++;
	energy = energy - int(random(4, 8));
	}
	
	do:
	
	if (isMouseOverOpaqueImage(allimg[WATER_INDEX], waterX, waterY, waterW, waterH)) {
	waterCounter++;
	energy = energy - int(random(4, 8));
	}
	Important detail for your current code
	
	Your button objects are written like this:
	
	const waterBtn = {

(isHover(starterBtn)) { starterCounter++; energy = energy - int(random(4, 8)); } if (isHover(saltBtn)) { saltCounter++; energy = energy - int(random(4, 8)); } if (isHover(waterBtn)) { waterCounter++; energy = energy - int(random(4, 8)); } if (isHover(flourBtn)) { flourCounter++; energy = energy - int(random(4, 8)); } } // ---------- // Keyboard input for instructions screen // ---------- // Provides keyboard-only navigation function pantryKeyPressed() { // ESC is a common “go back” key in games and apps if (keyCode === ESCAPE) { currentScreen = "home"; } // B key is an additional, explicit shortcut for “back” if (key === "b" || key === "B") { currentScreen = "home"; } } // ---------- // Button drawing helper (instructions screen) // ---------- // This function is only responsible for drawing the button. // It is kept separate so the visual style can be changed // without touching input or game logic. function drawPantryButton({ x, y, w, h, label }) { rectMode(CENTER); // Check whether the mouse is hovering over the button const hover = isHover({ x, y, w, h }); noStroke(); // Subtle colour change on hover for visual feedback fill(hover ? color(200, 200, 255, 200) : color(220, 220,

```
  x: width * 0.5,
  y: height / 3,
  w: 120,
  h: 180
};
```

But your helper above assumes x, y are the top-left corner of the image, because imageMode(CORNER) is active.

So for pixel detection to work cleanly, define image positions as actual top-left coordinates.

For example:

```
const waterBtn = {
  x: width / 2 - 60,
  y: height / 3,
  w: 120,
  h: 180,
  label: "Water",
};
```

because half the width is 60.

A version directly adapted to your style

You could rewrite your water item like this.

In drawPantry()

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const waterBtn = {
    x: width / 2 - 60,
    y: height / 3,
    w: 120,
    h: 180,
    label: "Water",
  };

  // draw the water jug PNG instead of relying on a rectangle
  image(allimg[7], waterBtn.x, waterBtn.y, waterBtn.w, waterBtn.h);

  text(waterCounter, waterBtn.x + waterBtn.w / 2, waterBtn.y + waterBtn.h + 20);
}
```

In pantryMousePressed()

```
function pantryMousePressed() {
  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };
```

255, 170)); // Draw the button shape rect(x, y, w, h, 12); // Draw the button text fill(0); textSize(26); textAlign(CENTER, CENTER); text(label, x, y);
}

```
  const waterBtn = {
    x: width / 2 - 60,
    y: height / 3,
    w: 120,
    h: 180,
  };

  if (isHover(backBtn)) {
    currentScreen = "home";
    return;
  }

  if (isMouseOverOpaqueImage(allimg[7], waterBtn.x, waterBtn.y, waterBtn.w,
waterBtn.h)) {
    waterCounter++;
    energy = energy - int(random(4, 8));
  }
}
```

If you want the actual area of the PNG

If you want to know the shape area in pixels, add this function:

```
function getOpaquePixelArea(img) {
  img.loadPixels();

  let count = 0;
  for (let i = 3; i < img.pixels.length; i += 4) {
    if (img.pixels[i] > 10) {
      count++;
    }
  }
  return count;
}
```

Then use:

```
let jugArea = getOpaquePixelArea(allimg[7]);
console.log(jugArea);
```

That gives the number of visible pixels in the original PNG.

If you scale the image when drawing, the displayed area changes roughly like this:

```
let displayedArea = jugArea * (drawW / img.width) * (drawH / img.height);
```

Best use for your game

For your pantry game, the most useful thing is probably not the area number itself. The most useful thing is:

- keep width/height for drawing
- use pixel-perfect click detection for irregular PNGs

	That way the player only clicks the actual jug shape, not the empty transparent space around it. Paste your drawButton() or isHover() code next and I'll show you exactly how to convert one of your pantry items fully into the PNG-based version.

Annika- what did i do wrong: // NOTE: Do NOT add setup() or draw() in this file // setup() and draw() live in main.js // This file only defines: // 1) drawPantry() → what the pantry screen looks like // 2) input handlers → how the player returns to the home screen // 3) helper functions specific to this screen // ---------- // Main draw function for pantry screen // ---------- // drawPantry() is called from main.js // only when currentScreen === "pantry" function drawPantry() { // Light neutral background background(240); imageMode(CORNER); image(allimg[28], 0, 0, width, height); // draw sourdough starter png image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140); // ---- INGREDIENTS ---- // flour const flourBtn = { w: 100, h: 100, x: width / 2, y: height / 3, label: "Flour", }; // water const waterBtn = { w: 120, h: 180, x: width * 0.5, y: height / 3, label: "Water", }; // sourdough starter const starterBtn = { w: 80, h: 140, x: width * 0.25, y: height / 3, label: "Starter", }; // salt const saltBtn = { w: 50, h: 100, x: width * 0.75, y: height / 2, label: "Salt", }; // Draw all buttons	A few things are wrong here.
	
	1) You are checking the starter twice
	
	You have both:
	
	`if (isHover(starterBtn)) {`
	`  starterCounter++;`
	`  energy = energy - int(random(4, 8));`
	`}`
	
	and also:
	
	`if (`
	`  isMouseOverOpaqueImage(`
	`  allimg[6],`
	`  starterBtn.x,`
	`  starterBtn.y,`
	`  starterBtn.w,`
	`  starterBtn.h,`
	`  )`
	`) {`
	`  starterCounter++;`
	`  energy = energy - int(random(4, 8));`
	`}`
	
	So one click can trigger both and add twice.
	
	Use only one of them for the starter. Since you want the irregular PNG, keep the isMouseOverOpaqueImage(...) one and remove the isHover(starterBtn) one.
	
	2) Your click area does not match where the image is drawn
	
	You draw the starter image here:
	
	`image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140);`
	
	But your starter button is defined as:
	
	`const starterBtn = {`
	`  x: width * 0.25,`
	`  y: height / 3,`
	`  w: 80,`
	`  h: 140,`

```
drawButton(waterBtn);
drawButton(starterBtn);
drawButton(saltBtn);
drawButton(flourBtn);
text(waterCounter,
waterBtn.x, waterBtn.y +
20); text(saltCounter,
saltBtn.x, saltBtn.y + 20);
text(starterCounter,
starterBtn.x, starterBtn.y +
20); text(flourCounter,
flourBtn.x, flourBtn.y + 20);
} // ------------------------------ //
Mouse input for instructions
screen //
------------------------------
function
isMouseOverOpaqueImage
(img, x, y, w, h) { // first:
check if mouse is inside the
image bounds if (mouseX <
x || mouseX > x + w ||
mouseY < y || mouseY > y
+ h) { return false; } //
convert mouse position on
screen to pixel position
inside original image let
imgX = floor(map(mouseX,
x, x + w, 0, img.width)); let
imgY = floor(map(mouseY,
y, y + h, 0, img.height)); //
keep within bounds imgX =
constrain(imgX, 0,
img.width - 1); imgY =
constrain(imgY, 0,
img.height - 1); // get pixel
color from image let c =
img.get(imgX, imgY); // c =
[r, g, b, a] let alphaValue =
c[3]; // treat only visible
pixels as clickable return
alphaValue > 10; } // Called
from main.js only when
currentScreen === "pantry"
function
pantryMousePressed() {
const backBtn = { x: width /
2, y: 560, w: 220, h: 70 };
const waterBtn = { x: width
* 0.5, y: height / 3, w: 120,
```

```
  label: "Starter",
};
```

Those are not the same position or size.

So your code is checking clicks in one place, but the image is shown somewhere else.

You need the click test coordinates to exactly match the image draw coordinates.

For example:

```
const starterBtn = {
  x: width / 2 - 160,
  y: height / 2 - 60,
  w: 140,
  h: 140,
  label: "Starter",
};
```

Then draw and test using that same object.

3) Your isMouseOverOpaqueImage() assumes x, y are top-left coordinates

This function checks bounds like this: if (mouseX < x || mouseX > x + w || mouseY < y || mouseY > y + h)

That means x and y must be the top-left corner.

But your button drawing helper uses: rectMode(CENTER);

So in other parts of your code, x and y are being treated like the center.
That mismatch breaks things.

For pixel-perfect image clicking, use:

imageMode(CORNER)

x, y = top-left

same x, y, w, h for both drawing and clicking

4) You call drawButton(...), but the function you defined is drawPantryButton(...)

In drawPantry() you wrote:

```
drawButton(waterBtn);
drawButton(starterBtn);
drawButton(saltBtn);
drawButton(flourBtn);
```

h: 180 }; const starterBtn = { x: width * 0.25, y: height / 3, w: 80, h: 140 }; const saltBtn = { x: width * 0.75, y: height / 2, w: 50, h: 100 }; const flourBtn = { x: width / 2, y: height / 3, w: 100, h: 100 }; // If the button is clicked, return to the home screen if (isHover(backBtn)) { currentScreen = "home"; return; } //checks if ingredients are clicked if (isHover(starterBtn)) { starterCounter++; energy = energy - int(random(4, 8)); } if (isMouseOverOpaqueImage (allimg[6], starterBtn.x, starterBtn.y, starterBtn.w, starterBtn.h,)) { starterCounter++; energy = energy - int(random(4, 8)); } if (isHover(saltBtn)) { saltCounter++; energy = energy - int(random(4, 8)); } if (isHover(waterBtn)) { waterCounter++; energy = energy - int(random(4, 8)); } if (isHover(flourBtn)) { flourCounter++; energy = energy - int(random(4, 8)); } } // ---------- // Keyboard input for instructions screen // ---------- // Provides keyboard-only navigation function pantryKeyPressed() { // ESC is a common “go back” key in games and apps if (keyCode === ESCAPE) { currentScreen = "home"; } // B key is an additional, explicit shortcut for “back” if (key === "b" || key === "B") { currentScreen = "home"; } } // ---------- //

But the function at the bottom is:

function drawPantryButton({ x, y, w, h, label }) {

So unless drawButton() exists somewhere else, that is a naming error.

It should either be:

drawPantryButton(waterBtn);
drawPantryButton(starterBtn);
drawPantryButton(saltBtn);
drawPantryButton(flourBtn);

or rename the function to drawButton.

5) You are using pixel detection on the starter image, but still drawing a rectangle button for it

Right now you draw:

- the PNG image separately
- a rectangle button separately

So visually, the player may see both the PNG and a box, or the box may not line up.

If you want the starter to be clicked by its PNG shape, do not also use a normal rectangle button for that same object unless that is intentional.

Clean fix for just the starter

Use this version.

In drawPantry()

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const starterBtn = {
    x: width / 2 - 160,
    y: height / 2 - 60,
    w: 140,
    h: 140,
    label: "Starter",
  };

  image(allimg[6], starterBtn.x, starterBtn.y, starterBtn.w, starterBtn.h);

  textAlign(CENTER, CENTER);
  text(starterCounter, starterBtn.x + starterBtn.w / 2, starterBtn.y + starterBtn.h + 20);
```

Button drawing helper (instructions screen) // ---------- // This function is only responsible for drawing the button. // It is kept separate so the visual style can be changed // without touching input or game logic. function drawPantryButton({ x, y, w, h, label }) { rectMode(CENTER); // Check whether the mouse is hovering over the button const hover = isHover({ x, y, w, h }); noStroke(); // Subtle colour change on hover for visual feedback fill(hover ? color(200, 200, 255, 200) : color(220, 220, 255, 170)); // Draw the button shape rect(x, y, w, h, 12); // Draw the button text fill(0); textSize(26); textAlign(CENTER, CENTER); text(label, x, y); }	}
	In pantryMousePressed()
	function pantryMousePressed() {
	  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };
	
	  const starterBtn = {
	  x: width / 2 - 160,
	  y: height / 2 - 60,
	  w: 140,
	  h: 140,
	  };
	
	  if (isHover(backBtn)) {
	  currentScreen = "home";
	  return;
	  }
	
	  if (
	  isMouseOverOpaqueImage(
	  allimg[6],
	  starterBtn.x,
	  starterBtn.y,
	  starterBtn.w,
	  starterBtn.h
	  )
	  ) {
	  starterCounter++;
	  energy -= int(random(4, 8));
	  }
	}
	Main problems in one line
	The biggest mistakes are:
	● checking the starter twice
	● using hitbox coordinates that do not match the image position
	● mixing center-based coordinates with top-left-based coordinates
	● calling drawButton() when your function is named drawPantryButton()
	
	Paste your isHover() function and I can point out whether that part also has a center/corner mismatch.

Annika- how would i make it so that the sourdough starter png goes darker when hovering	You're using p5.js, so the easiest way to make the PNG darker on hover is to use the tint() function. tint() multiplies the color of an image. Lower values make the image darker. 1. Detect hover using your existing function You already have: isMouseOverOpaqueImage(img, x, y, w, h) We can reuse it.

2. Apply tint when hovering
Wrap the image draw with a hover check.

Example for your starter image.

```
const starterBtn = {
  x: width / 2 - 160,
  y: height / 2 - 60,
  w: 140,
  h: 140,
};

const hoveringStarter = isMouseOverOpaqueImage(
  allimg[6],
  starterBtn.x,
  starterBtn.y,
  starterBtn.w,
  starterBtn.h
);

// darken image if hovering
if (hoveringStarter) {
  tint(150);   // darker
} else {
  tint(255);   // normal
}

image(allimg[6], starterBtn.x, starterBtn.y, starterBtn.w, starterBtn.h);

// reset tint so other images aren't affected
noTint();
```

3. What the numbers mean

tint(value) uses 0–255 brightness:

tint value	result
255	normal
200	slightly darker
150	clearly darker
100	very dark

4. Where this goes in your code

Inside drawPantry(), replace:

```
image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140);
```

with the hover version above.

5. Optional (makes the game feel better)
You can add a slight scale-up effect when hovering:

	let scale = hoveringStarter ? 1.05 : 1;
	
	image(
	  allimg[6],
	  starterBtn.x,
	  starterBtn.y,
	  starterBtn.w * scale,
	  starterBtn.h * scale
	);
	
	This gives a “button feel” in addition to the darkening.
	
	✅ Result:
	When the mouse is over the visible pixels of the sourdough PNG, it will darken slightly.
	
	If you want, I can also show you a much cleaner pantry structure (about 10 lines shorter) where each ingredient automatically handles:
	● hover
	● darkening
	● clicking
	● counters
	
	This will make the pantry code easier to maintain as you add ingredients.

// NOTE: Do NOT add setup() or draw() in this file // setup() and draw() live in main.js // This file only defines: // 1) drawPantry() → what the pantry screen looks like // 2) input handlers → how the player returns to the home screen // 3) helper functions specific to this screen // ---------- // Main draw function for pantry screen // ---------- // drawPantry() is called from main.js // only when currentScreen === "pantry" function drawPantry() { // Light neutral background background(240); imageMode(CORNER); image(allimg[28], 0, 0, width, height); // draw sourdough starter png	Add the hover check inside drawPantry() and use tint() before drawing the starter image.
	
	In your current code, replace this:
	
	// draw sourdough starter png
	image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140);
	
	with this:
	
	// sourdough starter image position
	const starterImg = {
	  x: width / 2 - 160,
	  y: height / 2 - 60,
	  w: 140,
	  h: 140,
	};
	
	// check hover on visible part of png
	const hoveringStarter = isMouseOverOpaqueImage(
	  allimg[6],
	  starterImg.x,
	  starterImg.y,
	  starterImg.w,
	  starterImg.h

image(allimg[6], width / 2 - 160, height / 2 - 60, 140, 140); // ---- INGREDIENTS ---- // flour const flourBtn = { w: 100, h: 100, x: width / 2, y: height / 3, label: "Flour", }; // water const waterBtn = { w: 120, h: 180, x: width * 0.5, y: height / 3, label: "Water", }; // sourdough starter const starterBtn = { w: 80, h: 140, x: width * 0.25, y: height / 3, label: "Starter", }; // salt const saltBtn = { w: 50, h: 100, x: width * 0.75, y: height / 2, label: "Salt", }; // Draw all buttons drawButton(waterBtn); drawButton(starterBtn); drawButton(saltBtn); drawButton(flourBtn); text(waterCounter, waterBtn.x, waterBtn.y + 20); text(saltCounter, saltBtn.x, saltBtn.y + 20); text(starterCounter, starterBtn.x, starterBtn.y + 20); text(flourCounter, flourBtn.x, flourBtn.y + 20); } // ---------- // Mouse input for instructions screen // ---------- function isMouseOverOpaqueImage (img, x, y, w, h) { // first: check if mouse is inside the image bounds if (mouseX < x || mouseX > x + w || mouseY < y || mouseY > y + h) { return false; } // convert mouse position on screen to pixel position inside original image let imgX = floor(map(mouseX, x, x + w, 0, img.width)); let imgY = floor(map(mouseY, y, y + h, 0, img.height)); // keep within bounds imgX =

```
);

// darken when hovering
if (hoveringStarter) {
  tint(150);
} else {
  tint(255);
}

image(allimg[6], starterImg.x, starterImg.y, starterImg.w, starterImg.h);
noTint();
```

So the top of drawPantry() would look like:

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const starterImg = {
    x: width / 2 - 160,
    y: height / 2 - 60,
    w: 140,
    h: 140,
  };

  const hoveringStarter = isMouseOverOpaqueImage(
    allimg[6],
    starterImg.x,
    starterImg.y,
    starterImg.w,
    starterImg.h
  );

  if (hoveringStarter) {
    tint(150);
  } else {
    tint(255);
  }

  image(allimg[6], starterImg.x, starterImg.y, starterImg.w, starterImg.h);
  noTint();

  // rest of your pantry code...
}
```

Two important fixes in your file too:

First, you are calling:

constrain(imgX, 0, img.width - 1); imgY = constrain(imgY, 0, img.height - 1); // get pixel color from image let c = img.get(imgX, imgY); // c = [r, g, b, a] let alphaValue = c[3]; // treat only visible pixels as clickable return alphaValue > 10; } // Called from main.js only when currentScreen === "pantry" function pantryMousePressed() { const backBtn = { x: width / 2, y: 560, w: 220, h: 70 }; const waterBtn = { x: width * 0.5, y: height / 3, w: 120, h: 180 }; const starterBtn = { x: width / 2 - 160, y: height / 2 - 60, w: 140, h: 140 }; const saltBtn = { x: width * 0.75, y: height / 2, w: 50, h: 100 }; const flourBtn = { x: width / 2, y: height / 3, w: 100, h: 100 }; // If the button is clicked, return to the home screen if (isHover(backBtn)) { currentScreen = "home"; return; } if (isMouseOverOpaqueImage(allimg[6], starterBtn.x, starterBtn.y, starterBtn.w, starterBtn.h,)) { starterCounter++; energy = energy - int(random(4, 8)); } if (isHover(saltBtn)) { saltCounter++; energy = energy - int(random(4, 8)); } if (isHover(waterBtn)) { waterCounter++; energy = energy - int(random(4, 8)); } if (isHover(flourBtn)) { flourCounter++; energy = energy - int(random(4, 8)); } } // ---------- // Keyboard input for instructions screen // ---------- //

drawButton(waterBtn);

but the function you actually defined is:

function drawPantryButton(...)

So either change the calls to:

drawPantryButton(waterBtn);
drawPantryButton(starterBtn);
drawPantryButton(saltBtn);
drawPantryButton(flourBtn);

or rename the function.

Second, in drawPantryButton() you have:

text(label, x, y);

With rectMode(CORNER), that puts the text at the top-left corner of the box, not the center. Change it to:

text(label, x + w / 2, y + h / 2);

So that function should be:

```
function drawPantryButton({ x, y, w, h, label }) {
  rectMode(CORNER);

  const hover = isHover({ x, y, w, h });

  noStroke();
  fill(hover ? color(200, 200, 255, 200) : color(220, 220, 255, 170));
  rect(x, y, w, h, 12);

  fill(0);
  textSize(26);
  textAlign(CENTER, CENTER);
  text(label, x + w / 2, y + h / 2);
}
```

If you want a softer darkening effect, use tint(200) instead of tint(150).

Provides keyboard-only navigation function pantryKeyPressed() { // ESC is a common “go back” key in games and apps if (keyCode === ESCAPE) { currentScreen = "home"; } // B key is an additional, explicit shortcut for “back” if (key === "b" || key === "B") { currentScreen = "home"; } } // ---------- // Button drawing helper (instructions screen) // ---------- // This function is only responsible for drawing the button. // It is kept separate so the visual style can be changed // without touching input or game logic. function drawPantryButton({ x, y, w, h, label }) { rectMode(CORNER); // Check whether the mouse is hovering over the button const hover = isHover({ x, y, w, h }); noStroke(); // Subtle colour change on hover for visual feedback fill(hover ? color(200, 200, 255, 200) : color(220, 220, 255, 170)); // Draw the button shape rect(x, y, w, h, 12); // Draw the button text fill(0); textSize(26); textAlign(CENTER, CENTER); text(label, x, y); }	
Annika- how can i make this code efficient so that the sour dough starter and salt have the same main functions so that i dont have to have a separate constant for each ingredient	Yes — the clean way is to put your pantry items into one array of ingredient objects, then loop through them for: drawing hover darkening clicking

counters

Right now you are repeating the same structure for each ingredient. The duplication is happening because your data is spread across separate constants.

Better structure

Make one array like this:

```
function getPantryIngredients() {
  return [
    {
      name: "Starter",
      img: allimg[6],
      x: width / 2 - 160,
      y: height / 2 - 60,
      w: 140,
      h: 140,
      counterName: "starterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Salt",
      img: allimg[5],
      x: width / 2 - 110,
      y: height / 2 + 170,
      w: 105,
      h: 105,
      counterName: "saltCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Water",
      x: width * 0.5,
      y: height / 3,
      w: 120,
      h: 180,
      label: "Water",
      counterName: "waterCounter",
      useOpaqueHitbox: false,
    },
    {
      name: "Flour",
      x: width / 2,
      y: height / 3,
      w: 100,
      h: 100,
      label: "Flour",
      counterName: "flourCounter",
```

```
      useOpaqueHitbox: false,
    },
  ];
}
```

This makes each ingredient follow the same format.

Then in drawPantry()

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    if (ingredient.img) {
      const hovering = isMouseOverOpaqueImage(
        ingredient.img,
        ingredient.x,
        ingredient.y,
        ingredient.w,
        ingredient.h
      );

      tint(hovering ? 150 : 255);
      image(ingredient.img, ingredient.x, ingredient.y, ingredient.w, ingredient.h);
      noTint();
    } else {
      drawPantryButton(ingredient);
    }

    drawIngredientCounter(ingredient);
  }
}
```

Add a counter helper

```
function drawIngredientCounter(ingredient) {
  fill(0);
  textSize(20);
  textAlign(CENTER, CENTER);

  let count = 0;

  if (ingredient.counterName === "starterCounter") count = starterCounter;
  if (ingredient.counterName === "saltCounter") count = saltCounter;
  if (ingredient.counterName === "waterCounter") count = waterCounter;
  if (ingredient.counterName === "flourCounter") count = flourCounter;

  text(count, ingredient.x + ingredient.w / 2, ingredient.y + ingredient.h + 20);
}
```

Then in pantryMousePressed()

```
function pantryMousePressed() {
  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };

  if (isHover(backBtn)) {
    currentScreen = "home";
    return;
  }

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    let clicked = false;

    if (ingredient.useOpaqueHitbox && ingredient.img) {
      clicked = isMouseOverOpaqueImage(
        ingredient.img,
        ingredient.x,
        ingredient.y,
        ingredient.w,
        ingredient.h
      );
    } else {
      clicked = isHover(ingredient);
    }

    if (clicked) {
      incrementIngredientCounter(ingredient.counterName);
      energy -= int(random(4, 8));
    }
  }
}
```

Add one helper for counters

```
function incrementIngredientCounter(counterName) {
  if (counterName === "starterCounter") starterCounter++;
  if (counterName === "saltCounter") saltCounter++;
  if (counterName === "waterCounter") waterCounter++;
  if (counterName === "flourCounter") flourCounter++;
}
```

Fix your button drawing helper

Your current drawPantryButton() has:

```
text(label, x, y);
```

That puts text at the top-left. Use this instead:

```
function drawPantryButton({ x, y, w, h, label }) {
  rectMode(CORNER);
```

```
  const hover = isHover({ x, y, w, h });

  noStroke();
  fill(hover ? color(200, 200, 255, 200) : color(220, 220, 255, 170));
  rect(x, y, w, h, 12);

  fill(0);
  textSize(26);
  textAlign(CENTER, CENTER);
  text(label, x + w / 2, y + h / 2);
}
```

Why this is better

Now starter and salt use the same system because:

both are just entries in the same array

both use the same draw loop

both use the same click loop

both use the same counter logic

So instead of writing:

```
const starterImg = ...
const saltImg = ...
const starterBtn = ...
const saltBtn = ...
```

you just add another object to the array.

One important problem in your current code

You also currently have different positions for some ingredients in draw vs click logic. That causes bugs.

With the array method, each ingredient has one source of truth for:

x

y

w

h

image

label

click method

That makes it much easier to keep correct.

Here is the full cleaned version together:

```
function getPantryIngredients() {
  return [
    {
      name: "Starter",
      img: allimg[6],
      x: width / 2 - 160,
      y: height / 2 - 60,
      w: 140,
      h: 140,
      counterName: "starterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Salt",
      img: allimg[5],
      x: width / 2 - 110,
      y: height / 2 + 170,
      w: 105,
      h: 105,
      counterName: "saltCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Water",
      x: width * 0.5,
      y: height / 3,
      w: 120,
      h: 180,
      label: "Water",
      counterName: "waterCounter",
      useOpaqueHitbox: false,
    },
    {
      name: "Flour",
      x: width / 2,
      y: height / 3,
      w: 100,
      h: 100,
      label: "Flour",
      counterName: "flourCounter",
      useOpaqueHitbox: false,
    },
  ];
```

```
}

function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    if (ingredient.img) {
      const hovering = isMouseOverOpaqueImage(
        ingredient.img,
        ingredient.x,
        ingredient.y,
        ingredient.w,
        ingredient.h
      );

      tint(hovering ? 150 : 255);
      image(ingredient.img, ingredient.x, ingredient.y, ingredient.w, ingredient.h);
      noTint();
    } else {
      drawPantryButton(ingredient);
    }

    drawIngredientCounter(ingredient);
  }
}

function pantryMousePressed() {
  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };

  if (isHover(backBtn)) {
    currentScreen = "home";
    return;
  }

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    let clicked = false;

    if (ingredient.useOpaqueHitbox && ingredient.img) {
      clicked = isMouseOverOpaqueImage(
        ingredient.img,
        ingredient.x,
        ingredient.y,
        ingredient.w,
        ingredient.h
```

```
      );
    } else {
      clicked = isHover(ingredient);
    }

    if (clicked) {
      incrementIngredientCounter(ingredient.counterName);
      energy -= int(random(4, 8));
    }
  }
}

function incrementIngredientCounter(counterName) {
  if (counterName === "starterCounter") starterCounter++;
  if (counterName === "saltCounter") saltCounter++;
  if (counterName === "waterCounter") waterCounter++;
  if (counterName === "flourCounter") flourCounter++;
}

function drawIngredientCounter(ingredient) {
  let count = 0;

  if (ingredient.counterName === "starterCounter") count = starterCounter;
  if (ingredient.counterName === "saltCounter") count = saltCounter;
  if (ingredient.counterName === "waterCounter") count = waterCounter;
  if (ingredient.counterName === "flourCounter") count = flourCounter;

  fill(0);
  textSize(20);
  textAlign(CENTER, CENTER);
  text(count, ingredient.x + ingredient.w / 2, ingredient.y + ingredient.h + 20);
}

function drawPantryButton({ x, y, w, h, label }) {
  rectMode(CORNER);

  const hover = isHover({ x, y, w, h });

  noStroke();
  fill(hover ? color(200, 200, 255, 200) : color(220, 220, 255, 170));
  rect(x, y, w, h, 12);

  fill(0);
  textSize(26);
  textAlign(CENTER, CENTER);
  text(label, x + w / 2, y + h / 2);
}
```

The next improvement after this would be to store the actual counter value inside the object too, but that depends on how the rest of your game is organized.

Annika- https://github.com/annikazamora/a2_annika it breaks when i click pantry	The pantry screen is breaking because drawPantry() calls a function that is not in the repo version of pantry.js: isMouseOverOpaqueImage(...). You can see drawPantry() and pantryMousePressed() both call it in pantry.js, but the file ends after drawPantryButton(...), so that helper never gets defined.
	
	That means when homeMousePressed() sets currentScreen = "pantry", the next frame runs drawPantry(), and JavaScript hits a missing-function error. The pantry navigation itself is fine: home.js does send pantry clicks to "pantry", and main.js does route that screen to drawPantry().
	
	Paste this function back into pantry.js, above pantryMousePressed() or anywhere in the file:
	
	`function isMouseOverOpaqueImage(img, x, y, w, h) {`
	`  if (mouseX < x || mouseX > x + w || mouseY < y || mouseY > y + h) {`
	`  return false;`
	`  }`
	
	`  let imgX = floor(map(mouseX, x, x + w, 0, img.width));`
	`  let imgY = floor(map(mouseY, y, y + h, 0, img.height));`
	
	`  imgX = constrain(imgX, 0, img.width - 1);`
	`  imgY = constrain(imgY, 0, img.height - 1);`
	
	`  let c = img.get(imgX, imgY);`
	`  let alphaValue = c[3];`
	
	`  return alphaValue > 10;`
	`}`
	
	There is also a second issue to clean up after that: your shared isHover() helper in main.js assumes buttons use rectMode(CENTER), but drawPantryButton() uses rectMode(CORNER). That mismatch can make pantry hover/click boxes feel off even after the crash is fixed.
	
	So the immediate fix is:
	restore isMouseOverOpaqueImage(...)
	test pantry again then decide whether pantry buttons should use center-based coordinates like the rest of the project, or switch pantry hit detection fully to corner-based objects
	
	If you want, I can rewrite your entire pantry.js into one corrected version that matches the rest of your game's coordinate system.

Annika- apply the salt and starter concepts to the flour and water	Yes — make water and flour image objects too, the same way as starter and salt. Right now, starter and salt are using: img

useOpaqueHitbox: true

image drawing with darken-on-hover

pixel-based clicking

Water and flour are still using rectangle buttons. So you just need to convert them into image ingredients too.

Replace getPantryIngredients() with this

Use the correct image indexes for your flour and water PNGs in allimg[...].

```
function getPantryIngredients() {
  return [
    {
      name: "Starter",
      img: allimg[6],
      x: width / 2 - 160,
      y: height / 2 - 60,
      w: 140,
      h: 140,
      counterName: "starterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Salt",
      img: allimg[5],
      x: width / 2 - 110,
      y: height / 2 + 170,
      w: 105,
      h: 105,
      counterName: "saltCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Water",
      img: allimg[7], // change if your water image is a different index
      x: width / 2 + 40,
      y: height / 2 - 40,
      w: 120,
      h: 180,
      counterName: "waterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Flour",
      img: allimg[4], // change if your flour image is a different index
      x: width / 2 + 180,
      y: height / 2 + 120,
```

```
      w: 100,
      h: 100,
      counterName: "flourCounter",
      useOpaqueHitbox: true,
    },
  ];
}
```

Then simplify drawPantry()

Since now every ingredient is image-based, you do not need drawPantryButton() for these pantry items.

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    const hovering = isMouseOverOpaqueImage(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h
    );

    tint(hovering ? 150 : 255);
    image(ingredient.img, ingredient.x, ingredient.y, ingredient.w, ingredient.h);
    noTint();

    drawIngredientCounter(ingredient);
  }
}
```

Then simplify pantryMousePressed()

Because all ingredients now use image hitboxes:

```
function pantryMousePressed() {
  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };

  if (isHover(backBtn)) {
    currentScreen = "home";
    return;
  }

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
```

```
    const clicked = isMouseOverOpaqueImage(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h
    );

    if (clicked) {
      incrementIngredientCounter(ingredient.counterName);
      energy -= int(random(4, 8));
    }
  }
}
```

Your full cleaned version

```
function getPantryIngredients() {
  return [
    {
      name: "Starter",
      img: allimg[6],
      x: width / 2 - 160,
      y: height / 2 - 60,
      w: 140,
      h: 140,
      counterName: "starterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Salt",
      img: allimg[5],
      x: width / 2 - 110,
      y: height / 2 + 170,
      w: 105,
      h: 105,
      counterName: "saltCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Water",
      img: allimg[7], // change if needed
      x: width / 2 + 40,
      y: height / 2 - 40,
      w: 120,
      h: 180,
      counterName: "waterCounter",
      useOpaqueHitbox: true,
    },
    {
      name: "Flour",
      img: allimg[4], // change if needed
```

```
      x: width / 2 + 180,
      y: height / 2 + 120,
      w: 100,
      h: 100,
      counterName: "flourCounter",
      useOpaqueHitbox: true,
    },
  ];
}

function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    const hovering = isMouseOverOpaqueImage(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h
    );

    tint(hovering ? 150 : 255);
    image(ingredient.img, ingredient.x, ingredient.y, ingredient.w, ingredient.h);
    noTint();

    drawIngredientCounter(ingredient);
  }
}

function isMouseOverOpaqueImage(img, x, y, w, h) {
  if (mouseX < x || mouseX > x + w || mouseY < y || mouseY > y + h) {
    return false;
  }

  let imgX = floor(map(mouseX, x, x + w, 0, img.width));
  let imgY = floor(map(mouseY, y, y + h, 0, img.height));

  imgX = constrain(imgX, 0, img.width - 1);
  imgY = constrain(imgY, 0, img.height - 1);

  let c = img.get(imgX, imgY);
  let alphaValue = c[3];

  return alphaValue > 10;
}
```

```
function pantryMousePressed() {
  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };

  if (isHover(backBtn)) {
    currentScreen = "home";
    return;
  }

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    const clicked = isMouseOverOpaqueImage(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h
    );

    if (clicked) {
      incrementIngredientCounter(ingredient.counterName);
      energy -= int(random(4, 8));
    }
  }
}

function incrementIngredientCounter(counterName) {
  if (counterName === "starterCounter") starterCounter++;
  if (counterName === "saltCounter") saltCounter++;
  if (counterName === "waterCounter") waterCounter++;
  if (counterName === "flourCounter") flourCounter++;
}

function drawIngredientCounter(ingredient) {
  let count = 0;

  if (ingredient.counterName === "starterCounter") count = starterCounter;
  if (ingredient.counterName === "saltCounter") count = saltCounter;
  if (ingredient.counterName === "waterCounter") count = waterCounter;
  if (ingredient.counterName === "flourCounter") count = flourCounter;

  fill(0);
  textSize(20);
  textAlign(CENTER, CENTER);
  text(count, ingredient.x + ingredient.w / 2, ingredient.y + ingredient.h + 20);
}
```

One important note
You need to make sure:
allimg[7] is actually water

	allimg[4] is actually flour If those indexes are wrong, the pantry will still run, but the wrong images will show. The next cleanup step would be to remove counterName strings and store the counter values in a cleaner way.

Annika- how do i choose the width for the image, but have the height be auto/whatever matches the ratio of the actual image	In p5.js, if you want to set the width and keep the correct aspect ratio, compute the height using the image's original ratio.
	
	The formula is:
	height
	=
	width
	×
	originalHeight
	originalWidth
	height=width×
	originalWidth
	originalHeight
	
	So if your image is img, use:
	
	let w = 120;
	let h = w * (img.height / img.width);
	image(img, x, y, w, h);
	This keeps the image from stretching.
	
	How to apply this in your pantry system
	Right now your objects have fixed width and height:
	
	{
	  img: allimg[6],
	  x: width / 2 - 160,
	  y: height / 2 - 60,
	  w: 140,
	  h: 140,
	}
	
	Instead, store only width, then calculate height when drawing.
	Example:
	{
	  img: allimg[6],
	  x: width / 2 - 160,
	  y: height / 2 - 60,
	  w: 140
	}
	
	Then in drawPantry():
	
	for (let ingredient of ingredients) {

	let h = ingredient.w * (ingredient.img.height / ingredient.img.width);
	const hovering = isMouseOverOpaqueImage(
	ingredient.img,
	ingredient.x,
	ingredient.y,
	ingredient.w,
	h
	);
	
	tint(hovering ? 150 : 255);
	image(ingredient.img, ingredient.x, ingredient.y, ingredient.w, h);
	noTint();
	
	drawIngredientCounter({
	...ingredient,
	h: h
	});
	}
	Shortcut helper (cleaner)
	
	You can make a small helper function:
	function drawImageFitWidth(img, x, y, w) {
	let h = w * (img.height / img.width);
	image(img, x, y, w, h);
	return h;
	}
	
	Then use:
	let h = drawImageFitWidth(allimg[6], x, y, 140);
	Example for one ingredient
	let w = 140;
	let h = w * (allimg[6].height / allimg[6].width);
	
	image(allimg[6], width/2 - 160, height/2 - 60, w, h);
	Important note
	This only works after the image is loaded (which yours are in preload()).
	If img.width or img.height were 0, it would mean the image isn't loaded yet.
	
	If you want, I can also show you a 3-line pattern that automatically scales every pantry item to the same visual size while preserving ratios, which is really useful when all the PNGs have different original dimensions.

why does clicking the water jug take me back to the home page	Because the water jug is overlapping your back button area, and your code checks the back button first. In pantryMousePressed() you do this: if (isHover(backBtn)) { currentScreen = "home"; return;

	}
	
	So if the click lands anywhere inside the back button hitbox, the function exits before it ever checks the ingredients.
	
	Your back button is here:
	const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };
	And your water jug is here:
	
	{
	  x: width / 2 - 12,
	  y: height / 2 + 100,
	  w: 165,
	  h: 185,
	}
	
	If your canvas is around 600 px tall, the jug extends pretty far downward and can overlap the back button region.
	
	For example, the jug covers roughly:
	x: width/2 - 12 to width/2 + 153
	y: height/2 + 100 to height/2 + 285
	
	So on a 600 px-tall canvas, that is about:
	
	y: 400 to 585
	Your back button centered at y = 560 with height 70 covers about:
	y: 525 to 595
	Those overlap.
	
	Why it happens only with water
	Because the water jug is tall and placed low enough that its bottom overlaps the back button zone.
	
	Fix options
	Best fix: move the water jug up
	
	For example:
	
	{
	  name: "Water",
	  img: allimg[13],
	  x: width / 2 - 12,
	  y: height / 2 + 40,
	  w: 165,
	  h: 185,
	  counterName: "waterCounter",
	  useOpaqueHitbox: true,
	}
	Or move the back button lower

	If there is room on the screen.
	Or check ingredients before the back button
	That way if the player clicks on the visible water jug, the ingredient wins first:
	
	function pantryMousePressed() {
	  const backBtn = { x: width / 2, y: 560, w: 220, h: 70 };
	  const ingredients = getPantryIngredients();
	
	  for (let ingredient of ingredients) {
	  const clicked = isMouseOverOpaqueImage(
	  ingredient.img,
	  ingredient.x,
	  ingredient.y,
	  ingredient.w,
	  ingredient.h
	  );
	
	  if (clicked) {
	  incrementIngredientCounter(ingredient.counterName);
	  energy -= int(random(4, 8));
	  return;
	  }
	  }
	
	  if (isHover(backBtn)) {
	  currentScreen = "home";
	  return;
	  }
	}
	
	That is usually the safest choice when clickable objects might overlap.
	One more thing to verify
	Your backBtn uses isHover(backBtn), and that helper may be using center-based coordinates, while your ingredient hitboxes use corner-based coordinates. That is fine as long as you remember:
	back button: likely center-based
	
	ingredient images: definitely corner-based
	The actual bug here is the overlap plus the order of checks.
	The simplest fix is to move the water jug upward a bit and/or check ingredients before the back button.

Annika- https://github.com/annikazamora/a2_annika how can i fix the recipe book image being not centred when going from the pantry file	The recipe book is not centered because the drawing mode is changing between screens. In your home screen, the recipe / pantry hover images use: imageMode(CENTER); Example from your repo: imageMode(CENTER);

image(allimg[1], 325, 485, 420, 550);

So those images are positioned from their center.
But in pantry.js you switch the mode to:
imageMode(CORNER);
image(allimg[28], 0, 0, width, height);

Then when you change screens, p5 keeps the last imageMode that was set. So when the recipe book draws later, it is still using CORNER, not CENTER.
That makes it appear shifted to the side.

Fix
Set the image mode explicitly before drawing the recipe book image.
Wherever the recipe book image is drawn, add:
imageMode(CENTER);

Example:
imageMode(CENTER);
image(allimg[RECIPE_INDEX], width / 2, height / 2, 420, 550);
Even better fix (recommended)

Whenever you draw a background, reset the mode immediately after so it doesn't affect other screens.

In drawPantry():

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);
  imageMode(CORNER); // keep ingredients consistent
```

And in recipe / workbench drawing:
imageMode(CENTER);
Why this happens

p5.js drawing settings like:
imageMode()
rectMode()
textAlign()
tint()

are global states.
When one screen changes them, the next screen inherits them unless you reset them.

✅ Simplest rule:
At the top of every screen draw function, explicitly set the modes you need.

Example:

	function drawRecipeBook() {
	  imageMode(CENTER);
	  rectMode(CENTER);
	  textAlign(CENTER, CENTER);
	}
	
	If you want, I can also show a clean pattern used in games where each screen resets drawing modes automatically, which prevents these bugs completely.

Annika- I want to move where the text for the ingredient counters are, into the corner so it looks like a legend/menu that tells how many of each ingredient has been collected (clicked)	Right now the counter text is drawn under each ingredient image because of this line in drawIngredientCounter():
	text(
	  count,
	  ingredient.x + ingredient.w / 2,
	  ingredient.y + ingredient.h + 20,
	);
	
	To turn it into a legend/menu in the corner, you should:
	Stop drawing counters next to each ingredient
	Create a separate function that draws a legend box
	Call that function inside drawPantry()
	
	1 Remove the per-ingredient counter
	In drawPantry() remove this line:
	drawIngredientCounter(ingredient);
	
	Your loop becomes:
	for (let ingredient of ingredients) {
	  const hovering = isMouseOverOpaqueImage(
	  ingredient.img,
	  ingredient.x,
	  ingredient.y,
	  ingredient.w,
	  ingredient.h,
	  );
	
	  tint(hovering ? 150 : 255);
	  image(
	  ingredient.img,
	  ingredient.x,
	  ingredient.y,
	  ingredient.w,
	  ingredient.h,
	  );
	  noTint();
	}
	2 Create a legend function
	Add this new function at the bottom of the file:
	
	function drawIngredientLegend() {

```
  const x = 20;
  const y = 20;

  fill(255, 240);
  rect(x - 10, y - 10, 170, 120, 10);

  fill(0);
  textSize(16);
  textAlign(LEFT, TOP);

  text("Ingredients", x, y);
  text("Starter: " + starterCounter, x, y + 25);
  text("Salt: " + saltCounter, x, y + 45);
  text("Water: " + waterCounter, x, y + 65);
  text("Flour: " + flourCounter, x, y + 85);
}
```

This draws a menu-style panel in the top-left.

3 Call the legend inside drawPantry()
At the bottom of drawPantry(), add:

```
drawIngredientLegend();
```

So the full function ends like:

```
function drawPantry() {
  background(240);
  imageMode(CORNER);
  image(allimg[28], 0, 0, width, height);

  const ingredients = getPantryIngredients();

  for (let ingredient of ingredients) {
    const hovering = isMouseOverOpaqueImage(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h,
    );

    tint(hovering ? 150 : 255);
    image(
      ingredient.img,
      ingredient.x,
      ingredient.y,
      ingredient.w,
      ingredient.h,
    );
    noTint();
  }
```

	drawIngredientLegend();
	}
	4️⃣ Result
	Now you'll see a menu like this in the corner:
	Ingredients
	Starter: 2
	Salt: 1
	Water: 3
	Flour: 0
	instead of numbers floating under the ingredients.
	
	✅ Optional (looks nicer):
	I can also show you how to make the legend:
	look like a game UI panel
	include tiny ingredient icons
	automatically update from getPantryIngredients()
	which makes the code cleaner for bigger games.

Tristan Prompt 1: I will give you the code for my workbench that I was making before I need you change what the bowl looks likes and the workbench and the back ground I will give you the assets I want. I want 8.png to be what the bowl looks like before the ingredients are in it and when all the ingredients are in I want it to look like 9.png. I also what the workbench screen to look like 14.png and the background to look like 31.png I will paste the code for my workbench.	Answer:
	The images provide the assets. I'll update the workbench p5.js code to use the PNG images for the bowl (empty = 8.png, full = 9.png), the workbench table (14.png), and the background (31.png).
	
	Check the uploaded files
	Check the uploaded files
	Here's the updated workbench code with your image assets integrated. The key changes are:
	
	Background → draws 31.png (brick wall + wood floor)
	Workbench table → draws 14.png centered at the bottom
	Bowl → shows 8.png (empty) or 9.png (dough-filled) based on whether all ingredients are added
	Ingredient tokens and recipe card are repositioned to work naturally with the new layout
	javascript
	// ----------
	// WORKBENCH SCREEN
	// (Updated to use image assets)
	// ----------
	
	const BREAD_RECIPE = { water: 2, starter: 1, salt: 1 };
	
	const INGREDIENT_STYLES = {
	  water: { bg: [180, 220, 255], label: "WATER", emoji: "💧" },

```
  starter: { bg: [245, 225, 185], label: "STARTER", emoji: "🫙" },
  salt: { bg: [240, 240, 240], label: "SALT", emoji: "🧂" },
};

let wbIngredients = [];
let wbBowl = {};
let wbContents = {};
let wbDragging = null;
let wbMessage = "";
let wbMessageTimer = 0;

// Image assets — load these in your preload() function:
// imgBackground = loadImage('31.png');
// imgWorkbench  = loadImage('14.png');
// imgBowlEmpty  = loadImage('8.png');
// imgBowlFull   = loadImage('9.png');

function initWorkbench() {
  wbContents = {};
  wbDragging = null;
  wbMessage = "";
  wbMessageTimer = 0;

  // Bowl hitbox — centred on the workbench top surface
  // 14.png workbench: rendered ~560 wide, top surface ~y = height*0.38
  wbBowl = { x: width / 2, y: height * 0.42, w: 180, h: 120 };

  wbIngredients = [];
  const names = Object.keys(INGREDIENT_STYLES);
  names.forEach((name, i) => {
    wbIngredients.push({
      name,
      x: 80 + i * 120,
      y: height * 0.82,
      w: 95,
      h: 75,
      count: 3,
    });
  });
}

// ── helper: is every recipe requirement met? ──────────────────────────
```

```
function wbRecipeComplete() {
  return Object.entries(BREAD_RECIPE).every(
    ([name, needed]) => (wbContents[name] || 0) >= needed
  );
}

function drawWorkbench() {
  // 1. Background (31.png)
  if (typeof imgBackground !== "undefined") {
    image(imgBackground, 0, 0, width, height);
  } else {
    background(220, 210, 200);
  }

  // 2. Workbench table (14.png) — bottom-centred, roughly lower 55 % of canvas
  const wbW = width * 0.78;
  const wbH = wbW * (9 / 16); // approximate aspect ratio of the asset
  const wbX = (width - wbW) / 2;
  const wbY = height - wbH + height * 0.08;
  if (typeof imgWorkbench !== "undefined") {
    image(imgWorkbench, wbX, wbY, wbW, wbH);
  }

  // 3. Bowl — swap sprite based on whether recipe is complete
  const bowlW = 180;
  const bowlH = bowlW * (9 / 14); // approx ratio of the bowl PNGs
  const bowlX = wbBowl.x - bowlW / 2;
  const bowlY = wbBowl.y - bowlH / 2;
  const bowlImg = wbRecipeComplete()
    ? (typeof imgBowlFull !== "undefined" ? imgBowlFull : null)
    : (typeof imgBowlEmpty !== "undefined" ? imgBowlEmpty : null);

  if (bowlImg) {
    image(bowlImg, bowlX, bowlY, bowlW, bowlH);
  } else {
    // Fallback drawn bowl
    fill(232, 215, 195);
    stroke(150, 110, 70);
    strokeWeight(3);
    arc(wbBowl.x, wbBowl.y, wbBowl.w, wbBowl.h, 0, PI, CHORD);
    ellipse(wbBowl.x, wbBowl.y, wbBowl.w, wbBowl.h * 0.28);
    fill(120, 75, 30);
    noStroke();
```

```
    textSize(11);
    textAlign(CENTER, CENTER);
    text("drop ingredients here", wbBowl.x, wbBowl.y + wbBowl.h * 0.45);
  }

  // 4. Bowl contents overlay (emoji counters inside bowl)
  drawWbBowlContents();

  // 5. Recipe card
  drawWbRecipe();

  // 6. Ingredient tokens (bottom row)
  drawWbIngredients();

  // 7. Bake button
  drawWbBakeButton();

  // 8. Dragged token
  drawWbDragging();

  // 9. Toast message
  drawWbMessage();

  cursor(wbIsOverIngredient() ? HAND : ARROW);
}

// ── drawing helpers ──────────────
```

```
  noStroke();
  rect(x + 3, y + 4, w, h, 10);

  fill(empty ? color(200, 200, 200) : color(...style.bg));
  stroke(empty ? color(170, 170, 170) : color(0, 0, 0, 40));
  strokeWeight(1.5);
  rect(x, y, w, h, 10);

  noStroke();
  fill(empty ? 160 : 60);
  textAlign(CENTER, CENTER);
  textSize(22);
  text(style.emoji, x, y + 5);
  textSize(10);
  text(style.label, x, y - 20);

  if (!empty) {
    textSize(10);
    fill(80, 40, 10);
    text(`×${count}`, x + w / 2 - 12, y - h / 2 + 10);
  }

  rectMode(CORNER);
}

function drawWbBowlContents() {
  const { x, y } = wbBowl;
  const items = Object.entries(wbContents).filter(([, c]) => c > 0);
  if (items.length === 0) return;

  let ix = x - (items.length * 26) / 2 + 13;
  for (const [name, count] of items) {
    noStroke();
    textSize(14);
    textAlign(CENTER, CENTER);
    text(INGREDIENT_STYLES[name].emoji, ix, y - 8);
    fill(60, 30, 10);
    textSize(9);
    text(`×${count}`, ix, y + 7);
    ix += 26;
  }
}
```

```
function drawWbRecipe() {
  const x = width - 185, y = 100, w = 165, h = 175;

  fill(255, 248, 215);
  stroke(190, 150, 90);
  strokeWeight(2);
  rectMode(CORNER);
  rect(x, y, w, h, 8);

  fill(80, 40, 10);
  noStroke();
  textSize(13);
  textAlign(CENTER, TOP);
  text("📋 RECIPE", x + w / 2, y + 10);

  stroke(200, 170, 110);
  strokeWeight(1);
  line(x + 10, y + 30, x + w - 10, y + 30);
  noStroke();

  textSize(11);
  let ty = y + 38;
  for (const [name, needed] of Object.entries(BREAD_RECIPE)) {
    const have = wbContents[name] || 0;
    const done = have >= needed;
    fill(done ? color(50, 140, 50) : color(80, 40, 10));
    textAlign(LEFT, TOP);
    text(
      `${done ? "✓" : "○"} ${INGREDIENT_STYLES[name].emoji}
${INGREDIENT_STYLES[name].label}`,
      x + 10, ty
    );
    fill(done ? color(50, 140, 50) : color(180, 60, 60));
    textAlign(RIGHT, TOP);
    text(`${have}/${needed}`, x + w - 10, ty);
    ty += 40;
  }
}

function drawWbBakeButton() {
  const btn = wbGetBtn();
  const hover = isHover(btn);
  rectMode(CORNER);
```

```
  fill(hover ? color(90, 175, 65) : color(70, 150, 50));
  stroke(40, 100, 30);
  strokeWeight(2);
  rect(btn.x, btn.y, btn.w, btn.h, 10);
  fill(255);
  noStroke();
  textSize(15);
  textAlign(CENTER, CENTER);
  text("✓ BAKE BREAD!", btn.x + btn.w / 2, btn.y + btn.h / 2);
}

function drawWbDragging() {
  if (!wbDragging) return;
  drawWbToken(wbDragging.x, wbDragging.y, 90, 75, wbDragging.name,
wbDragging.count);
}

function drawWbMessage() {
  if (wbMessageTimer <= 0) return;
  wbMessageTimer--;
  const a = map(wbMessageTimer, 0, 60, 0, 255);
  fill(80, 40, 10, a);
  noStroke();
  textSize(15);
  textAlign(CENTER, CENTER);
  text(wbMessage, width / 2, height - 20);
}

// ── utilities
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

function wbGetBtn() {
  return { x: width / 2 - 85, y: height - 50, w: 170, h: 38 };
}

function wbIsOverIngredient() {
  for (const ing of wbIngredients) {
    if (
      ing.count > 0 &&
      mouseX > ing.x - ing.w / 2 && mouseX < ing.x + ing.w / 2 &&
      mouseY > ing.y - ing.h / 2 && mouseY < ing.y + ing.h / 2
    ) return true;
```

```
  }
  return false;
}

function wbInsideBowl(px, py) {
  const { x, y, w, h } = wbBowl;
  const dx = (px - x) / (w / 2);
  const dy = (py - y) / (h / 2 + 30);
  return dx * dx + dy * dy < 1.2;
}

// ── input handlers
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

function workbenchMousePressed() {
  for (const ing of wbIngredients) {
    if (
      ing.count > 0 &&
      mouseX > ing.x - ing.w / 2 && mouseX < ing.x + ing.w / 2 &&
      mouseY > ing.y - ing.h / 2 && mouseY < ing.y + ing.h / 2
    ) {
      wbDragging = { name: ing.name, x: mouseX, y: mouseY, count: ing.count,
source: ing };
      return;
    }
  }
  if (isHover(wbGetBtn())) wbCheckRecipe();
}

function workbenchMouseDragged() {
  if (wbDragging) { wbDragging.x = mouseX; wbDragging.y = mouseY; }
}

function workbenchMouseReleased() {
  if (!wbDragging) return;
  if (wbInsideBowl(mouseX, mouseY)) {
    const name = wbDragging.name;
    wbContents[name] = (wbContents[name] || 0) + 1;
    wbDragging.source.count--;
    wbMessage = `Added ${INGREDIENT_STYLES[name].emoji}
${INGREDIENT_STYLES[name].label}!`;
    wbMessageTimer = 80;
```

```
  }
  wbDragging = null;
}

function workbenchKeyPressed() {
  if (keyCode === ENTER) wbCheckRecipe();
}

function wbCheckRecipe() {
  const missing = [], excess = [];
  for (const [name, needed] of Object.entries(BREAD_RECIPE)) {
    const have = wbContents[name] || 0;
    if (have < needed) missing.push(INGREDIENT_STYLES[name].label);
    else if (have > needed) excess.push(INGREDIENT_STYLES[name].label);
  }
  if (missing.length === 0 && excess.length === 0) {
    currentScreen = "oven";
  } else if (missing.length > 0) {
    wbMessage = `Missing: ${missing.join(", ")}`;
    wbMessageTimer = 140;
  } else {
    wbMessage = `Too much: ${excess.join(", ")}`;
    wbMessageTimer = 140;
  }
}
```

One thing to add in your preload() function (if you don't have it already):

```javascript
function preload() {
  imgBackground = loadImage('31.png');
  imgWorkbench  = loadImage('14.png');
  imgBowlEmpty  = loadImage('8.png');
  imgBowlFull   = loadImage('9.png');
}
```

What changed:

drawWorkbench() now layers: background image → workbench table image → bowl image (swaps to the dough-filled version once all ingredients meet the recipe) → UI on top

The bowl hitbox (wbBowl) is repositioned to sit on the workbench countertop

Ingredient tokens moved to height * 0.82 so they sit below the workbench surface

All fallback drawing code is kept, so if images fail to load the game still works

Tristan Prompt 2: Is there a way you can take away the background bowl and put the one that fills up there instead	Answer: Looking at the screenshot I can see the issue — both bowls are drawing on top of each other. The empty bowl (allimg[8]) is always showing, and the full bowl (allimg[9]) is drawing on top of it as a smaller second bowl. The fix is simple — instead of always drawing allimg[8], just show one bowl that swaps between the two images: Check current bowl drawing code Check current bowl drawing code The code looks correct — it should only draw one bowl. The problem is that wbBowl position is set to height * 0.43 but the workbench table image is centred at height * 0.62, so the bowl is sitting too high and overlapping with the table image which already has a bowl drawn into it as part of the artwork. The background bowl you're seeing is part of the 14.png workbench image itself. I need to move your bowl sprite to sit on top of the counter at the right position, and size it to cover/replace the drawn bowl in the table image: Edited a file, read a file Edited a file, read a file Replace your workbench.js with this. The bowl in your screenshot was actually baked into the 14.png workbench artwork — so no code could remove it. What this update does instead is position your bowl sprite (8.png/9.png) directly on top of the counter at the right height to cover it. If it's still slightly off after testing, let me know and I can nudge the counterY value up or down to line it up perfectly.

Tristan Prompt 3:What would I change to make it go left a tad	Answer:
	Change this line:
	
	javascript
	wbBowl.x = width / 2;
	To something like:
	
	javascript
	wbBowl.x = width / 2 - 30;
	Increase the number to move it further left.

Tristan Prompt 4: Okay now I need to put some more images in for the ingredients and also add a new one. First is 11.png needs to be flour. 13.png needs to be water. 6.png is starter. 5.png is salt now include those in. these images are already in the file just like the bowls and background.	Answer:I need to update `INGREDIENT_STYLES` to include flour, and update `drawWbToken` to show the images instead of the coloured boxes. I also need to add flour to the recipe and ingredients list.
	Viewed a file, edited a file
	Viewed a file, edited a file
	
	Now update `drawWbToken` to show the image instead of the coloured box:
	
	Viewed a file, edited a file
	Viewed a file, edited a file
	
	Now space out the 4 tokens so they don't overlap:
	
	Edited a file, read a file
	Edited a file, read a file
	
	Replace your `workbench.js` with this. Here's what changed:
	
	● **Flour added** to the recipe (`flour: 3`) and ingredients list
	● **All 4 tokens** now show their PNG images — `11.png` flour, `13.png` water, `6.png` starter, `5.png` salt
	● Tokens go grey when count hits 0
	● Labels and count badges still show below/on the image
	● Tokens spaced out to fit all 4 across the bottom
	
	Make sure `5.png`, `6.png`, `11.png`, and `13.png` are in your `libraries/assets/images/` folder!
	
	// ----------
	// WORKBENCH SCREEN
	// ----------
	//
	// Image assets used (loaded in main.js preload via allimg[]):
	// allimg[31] → background (31.png)
	// allimg[14] → workbench table (14.png)
	// allimg[8] → empty bowl (8.png)
	// allimg[9] → bowl with dough (9.png)
	
	const BREAD_RECIPE = { flour: 3, water: 2, starter: 1, salt: 1 };
	
	const INGREDIENT_STYLES = {
	flour: { bg: [245, 225, 185], label: "FLOUR", emoji: "🌾", imgIndex: 11 },
	water: { bg: [180, 220, 255], label: "WATER", emoji: "💧", imgIndex: 13 },

```
  starter: { bg: [245, 225, 185], label: "STARTER", emoji: "🫙", imgIndex: 6  },
  salt:    { bg: [240, 240, 240], label: "SALT",     emoji: "🧂", imgIndex: 5  },
};

let wbIngredients  = [];
let wbBowl         = {};
let wbContents     = {};
let wbDragging     = null;
let wbMessage      = "";
let wbMessageTimer = 0;
let wbShowRecipe   = false;

function initWorkbench() {
  wbContents     = {};
  wbDragging     = null;
  wbMessage      = "";
  wbMessageTimer = 0;
  wbShowRecipe   = false;

  wbBowl = { x: width / 2, y: height * 0.43, w: 200, h: 130 };

  wbIngredients = [];
  const names = Object.keys(INGREDIENT_STYLES);
  names.forEach((name, i) => {
    const pantryCount = {
      flour:   flourCounter,
      water:   waterCounter,
      starter: starterCounter,
      salt:    saltCounter,
    };
    wbIngredients.push({
      name,
      x: 100 + i * 130,
      y: height * 0.8,
      w: 100,
      h: 100,
      count: pantryCount[name] || 0,
    });
  });
}

function wbRecipeComplete() {
  return Object.entries(BREAD_RECIPE).every(
```

```
    ([name, needed]) => (wbContents[name] || 0) >= needed
  );
}

function drawWorkbench() {
  // 1. Background
  imageMode(CORNER);
  if (allimg[31]) {
    image(allimg[31], 0, 0, width, height);
  } else {
    background(220, 210, 205);
  }

  // 2. Workbench table
  imageMode(CENTER);
  const wbImgW = width * 0.72;
  const wbImgH = wbImgW * (9 / 16);
  const wbImgY = height * 0.62;
  if (allimg[14]) {
    image(allimg[14], width / 2, wbImgY, wbImgW, wbImgH);
  }

  // 3. Bowl
  const counterY = wbImgY - wbImgH * 0.41;
  wbBowl.x = width / 2 - 15;
  wbBowl.y = counterY;
  wbBowl.w = 400;
  wbBowl.h = 260;

  imageMode(CENTER);
  const bowlImg = wbRecipeComplete() ? allimg[9] : allimg[8];
  if (bowlImg) {
    image(bowlImg, wbBowl.x, wbBowl.y, wbBowl.w, wbBowl.h);
  } else {
    _drawWbBowlFallback();
  }

  // 4. Bowl contents
  drawWbBowlContents();

  // 5. Recipe card (only if open)
  if (wbShowRecipe) drawWbRecipe();
```

```
  // 6. Ingredient tokens
  drawWbIngredients();

  // 7. Bake button
  drawWbBakeButton();

  // 8. Recipe toggle button
  drawWbRecipeBtn();

  // 9. Dragged token
  drawWbDragging();

  // 10. Toast message
  drawWbMessage();

  cursor(wbIsOverIngredient() ? HAND : ARROW);
}

function drawWbIngredients() {
  for (const ing of wbIngredients) {
    if (wbDragging && wbDragging.name === ing.name) continue;
    drawWbToken(ing.x, ing.y, ing.w, ing.h, ing.name, ing.count);
  }
}

function drawWbToken(x, y, w, h, name, count) {
  const style = INGREDIENT_STYLES[name];
  const empty = count <= 0;
  const img = allimg[style.imgIndex];

  // Bowl-coloured background box
  rectMode(CENTER);
  fill(empty ? color(210, 200, 200) : color(232, 185, 185));
  stroke(190, 140, 140);
  strokeWeight(1.5);
  rect(x, y, w + 10, h + 40, 12);
  noStroke();

  imageMode(CENTER);

  if (empty) {
    tint(150, 150, 150);
  } else {
```

```
    noTint();
  }

  if (img) {
    image(img, x, y - 10, w, h);
  } else {
    rectMode(CENTER);
    fill(empty ? color(200, 200, 200) : color(...style.bg));
    stroke(empty ? color(170, 170, 170) : color(0, 0, 0, 40));
    strokeWeight(1.5);
    rect(x, y, w, h, 10);
    noStroke();
    fill(empty ? 160 : 60);
    textAlign(CENTER, CENTER);
    textSize(22);
    text(style.emoji, x, y + 5);
    rectMode(CORNER);
  }

  noTint();

  if (!empty) {
    fill(80, 40, 10);
    noStroke();
    textSize(13);
    textAlign(CENTER, CENTER);
    text(`×${count}`, x + w / 2 - 8, y - h / 2 + 10);
  }

  fill(60, 30, 10);
  noStroke();
  textSize(11);
  textAlign(CENTER, CENTER);
  text(style.label, x, y + h / 2 + 10);

  rectMode(CORNER);
  imageMode(CORNER);
}

function drawWbBowlContents() {
  const { x, y } = wbBowl;
  const items = Object.entries(wbContents).filter(([, c]) => c > 0);
  if (items.length === 0) return;
```

```
  const iconSize = 40;
  const spacing = iconSize + 10;
  let ix = x - (items.length * spacing) / 2 + spacing / 2;

  imageMode(CENTER);
  for (const [name, count] of items) {
    const style = INGREDIENT_STYLES[name];
    const img = allimg[style.imgIndex];
    if (img) {
      image(img, ix, y - 10, iconSize, iconSize);
    }
    fill(60, 30, 10);
    noStroke();
    textSize(10);
    textAlign(CENTER, CENTER);
    text(`×${count}`, ix, y + 22);
    ix += spacing;
  }
  imageMode(CORNER);
}

function _drawWbBowlFallback() {
  const { x, y, w, h } = wbBowl;
  fill(0, 0, 0, 35);
  noStroke();
  ellipse(x, y + h * 0.5, w + 20, 18);
  fill(232, 215, 195);
  stroke(150, 110, 70);
  strokeWeight(3);
  arc(x, y, w, h, 0, PI, CHORD);
  ellipse(x, y, w, h * 0.28);
  fill(215, 198, 178, 150);
  noStroke();
  ellipse(x, y, w - 20, h * 0.2);
  fill(120, 75, 30);
  noStroke();
  textSize(11);
  textAlign(CENTER, CENTER);
  text("drop ingredients here", x, y + h * 0.58);
}

function drawWbRecipe() {
```

```
  const x = width - 200, y = 110, w = 165, h = 215;
  rectMode(CORNER);
  fill(255, 248, 215);
  stroke(190, 150, 90);
  strokeWeight(2);
  rect(x, y, w, h, 8);

  fill(80, 40, 10);
  noStroke();
  textSize(13);
  textAlign(CENTER, TOP);
  text("📋 RECIPE", x + w / 2, y + 10);

  stroke(200, 170, 110);
  strokeWeight(1);
  line(x + 10, y + 30, x + w - 10, y + 30);
  noStroke();

  textSize(11);
  let ty = y + 38;
  for (const [name, needed] of Object.entries(BREAD_RECIPE)) {
    const have = wbContents[name] || 0;
    const done = have >= needed;
    fill(done ? color(50, 140, 50) : color(80, 40, 10));
    textAlign(LEFT, TOP);
    text(
      `${done ? "✓" : "○"} ${INGREDIENT_STYLES[name].emoji}
${INGREDIENT_STYLES[name].label}`,
      x + 10, ty
    );
    fill(done ? color(50, 140, 50) : color(180, 60, 60));
    textAlign(RIGHT, TOP);
    text(`${have}/${needed}`, x + w - 10, ty);
    ty += 40;
  }
}

function drawWbBakeButton() {
  const btn = wbGetBtn();
  const hover = isHover(btn);

  rectMode(CENTER);
  fill(hover ? color(90, 175, 65) : color(70, 150, 50));
```

```
  stroke(40, 100, 30);
  strokeWeight(2);
  rect(btn.x, btn.y, btn.w, btn.h, 10);

  imageMode(CENTER);
  if (allimg[18]) {
    image(allimg[18], btn.x - btn.w / 2 + 30, btn.y, 45, 35);
  }

  fill(255);
  noStroke();
  textSize(14);
  textAlign(CENTER, CENTER);
  text("BAKE BREAD!", btn.x + 15, btn.y);

  rectMode(CORNER);
  imageMode(CORNER);
}

function drawWbRecipeBtn() {
  const btn = wbGetRecipeBtn();
  const hover = isHover(btn);
  rectMode(CORNER);
  fill(hover ? color(200, 170, 90) : color(175, 145, 70));
  stroke(130, 100, 40);
  strokeWeight(2);
  rect(btn.x, btn.y, btn.w, btn.h, 10);
  fill(255);
  noStroke();
  textSize(13);
  textAlign(CENTER, CENTER);
  text(
    wbShowRecipe ? "✕ CLOSE RECIPE" : "📋 VIEW RECIPE",
    btn.x + btn.w / 2,
    btn.y + btn.h / 2,
  );
  rectMode(CORNER);
}

function drawWbDragging() {
  if (!wbDragging) return;
  drawWbToken(wbDragging.x, wbDragging.y, 90, 75, wbDragging.name,
wbDragging.count);
```

```
}

function drawWbMessage() {
  if (wbMessageTimer <= 0) return;
  wbMessageTimer--;
  const a = map(wbMessageTimer, 0, 60, 0, 255);
  fill(80, 40, 10, a);
  noStroke();
  textSize(15);
  textAlign(CENTER, CENTER);
  text(wbMessage, width / 2, height - 30);
}

function wbGetBtn() {
  return { x: width / 2, y: height - 62, w: 220, h: 50 };
}

function wbGetRecipeBtn() {
  return { x: width - 175, y: height - 55, w: 160, h: 42 };
}

function wbIsOverIngredient() {
  for (const ing of wbIngredients) {
    if (
      ing.count > 0 &&
      mouseX > ing.x - ing.w / 2 && mouseX < ing.x + ing.w / 2 &&
      mouseY > ing.y - ing.h / 2 && mouseY < ing.y + ing.h / 2
    ) return true;
  }
  return false;
}



function workbenchMousePressed() {
  // Click an ingredient token to add one to the bowl
  for (const ing of wbIngredients) {
    if (
      ing.count > 0 &&
      mouseX > ing.x - ing.w / 2 && mouseX < ing.x + ing.w / 2 &&
      mouseY > ing.y - ing.h / 2 && mouseY < ing.y + ing.h / 2
    ) {
      wbContents[ing.name] = (wbContents[ing.name] || 0) + 1;
```

	``
	ing.count--;
	wbMessage = `Added ${INGREDIENT_STYLES[ing.name].emoji} ${INGREDIENT_STYLES[ing.name].label}!`;
	wbMessageTimer = 80;
	return;
	}
	}
	if (isHover(wbGetBtn())) wbCheckRecipe();
	if (isHover(wbGetRecipeBtn())) wbShowRecipe = !wbShowRecipe;
	}
	
	function workbenchMouseDragged() {}
	
	function workbenchMouseReleased() {}
	
	function workbenchKeyPressed() {
	if (keyCode === ENTER) wbCheckRecipe();
	}
	
	function wbCheckRecipe() {
	const missing = [], excess = [];
	for (const [name, needed] of Object.entries(BREAD_RECIPE)) {
	const have = wbContents[name] || 0;
	if (have < needed) missing.push(INGREDIENT_STYLES[name].label);
	else if (have > needed) excess.push(INGREDIENT_STYLES[name].label);
	}
	if (missing.length === 0 && excess.length === 0) {
	currentScreen = "oven";
	} else if (missing.length > 0) {
	wbMessage = `Missing: ${missing.join(", ")}`;
	wbMessageTimer = 140;
	} else {
	wbMessage = `Too much: ${excess.join(", ")}`;
	wbMessageTimer = 140;
	}
	}
	``

Tristan Prompt 5:it seems you edited the recipe instructions as the background was shifted and some of the text. I want players to have to click to	Answer:Looking at the code, `drawWbRecipe()` is always drawing the recipe card on screen. I need to add a toggle so it only shows when the player clicks the recipe button in the navbar. The recipe card is already accessible via the navbar "RECIPE INSTRUCTIONS" button which takes you to a separate screen — so the simplest fix is to just **remove**

open the reciepe instructions and once they do that then it shows recipe on the screen.	**`drawWbRecipe()` from the workbench entirely** and let the existing navbar button handle it. That way the recipe card won't be cluttering the workbench screen. Remove recipe card from workbench draw and add toggle button instead Remove recipe card from workbench draw and add toggle button instead Now add the toggle variable and a button to open/close it: Edited 2 files, viewed a file Edited 2 files, viewed a file Now reset it in `initWorkbench` and add a recipe toggle button to the draw, and wire up the click: Reset wbShowRecipe on init Reset wbShowRecipe on init Now add the recipe toggle button to the draw function and wire it up to the mouse: Add recipe toggle button to draw and to mouse handler Add recipe toggle button to draw and to mouse handler Now add the button drawing function and update `workbenchMousePressed` to handle it: Add drawWbRecipeBtn function Add drawWbRecipeBtn function Now wire the click up in `workbenchMousePressed`: Edited a file, read a file Edited a file, read a file Here's what changed: ● The recipe card is **hidden by default** when you enter the workbench ● A **"📋 VIEW RECIPE"** button now sits in the bottom right corner ● Clicking it toggles the recipe card open/closed, and the button changes to **"✕ CLOSE RECIPE"** when open ● The recipe card will overlay the screen just like before, but only when the player asks for it
Tristan Prompt 6:can you revert what happens when	Answer: Looking at the screenshot, the recipe screen is overlapping with a background image in a weird way. The issue is in `recipe.js` — it's using

you press receipe instructions to the way it was before. this picture shows how it is messed up	`allimg[4]` and `allimg[3]` but those images aren't loading correctly. Let me check the original recipe.js and revert it to a clean version:
	Read original recipe.js
	Read original recipe.js
	
	The original `recipe.js` is fine — the problem is that `allimg[3]` (the recipe book image) is loading but it's overlapping the text. This is exactly how it was originally, so nothing in `recipe.js` changed. The issue is that `allimg[4]` and `allimg[3]` need to exist as `3.png` and `4.png` in your `libraries/assets/images/` folder.
	
	Also `recipeMousePressed` sends to `"start"` which doesn't exist — it should be `"home"`. Let me fix that:
	
	Created a file, read a file
	Created a file, read a file
	
	Replace your `recipe.js` with this. The main things to check:
	
	• Make sure `3.png` and `4.png` exist in `libraries/assets/images/` — those are the recipe book and background images. If they're missing the text will still show but without the images behind it
	• The screen now correctly returns to `"home"` when clicked instead of `"start"` which didn't exist
	
	function drawRecipe() {
	  background(235, 223, 226);
	
	  imageMode(CENTER);
	  if (allimg[4]) image(allimg[4], width / 2, height / 2, width, height);
	  if (allimg[3]) image(allimg[3], width / 2, 440, 1200, 650);
	
	  fill(0);
	  textAlign(LEFT, CENTER);
	  textSize(25);
	
	  // Game instructions
	  text("Today we are making my", 240, 300);
	  text("favourite sourdough bread!", 240, 335);
	  text("We have 3 orders,", 240, 400);
	  text("but we might not be able", 240, 435);
	  text("to make them all today.", 240, 470);
	  text("Balancing our energy is", 240, 535);
	  text("just as important as", 240, 570);

```
  text("filling orders!",            240, 605);

  // Ingredients
  text("Collect the following",       760, 200);
  text("items from the pantry.",      760, 235);
  text("Ingredients:",                760, 290);
  text("- Flour: "   + flourCounter   + "/3", 760, 340);
  text("- Water: "   + waterCounter   + "/2", 760, 390);
  text("- Starter: " + starterCounter + "/1", 760, 440);
  text("- Salt: "    + saltCounter    + "/1", 760, 490);

  // Instructions
  text("Combine all the ingredients", 760, 555);
  text("on the workbench, then",      760, 590);
  text("bake the dough in the oven.", 760, 625);
}

function recipeMousePressed() {
  currentScreen = "home";
}

function recipeKeyPressed() {
  if (key === "r" || key === "R") {
    currentScreen = "home";
  }
}
```